



Offchain Labs Stylus SDK

Security Assessment

April 8, 2026

Prepared for:
Offchain Labs

Prepared by: **Samuel Moelius, Simone Monica, Kevin Valerio, and Jaime Iglesias**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	8
Project Targets	9
Project Coverage	10
Automated Testing	12
Codebase Maturity Evaluation	13
Summary of Findings	15
Detailed Findings	18
1. Reliance on vulnerable brotli2 package	18
2. When a contract is being verified, its build script runs as root	20
3. Contract build scripts have network access in Docker container	22
4. verify_create_deployment always returns Ok()	24
5. run_in_docker_container does not propagate exit codes	26
6. Manual termination does not kill child process during contract deployment	28
7. Deploy and Verify commands do not respect user options when run in a reproducible way	30
8. Deploy command does not call the constructor of a contract with no arguments	33
9. Verify command does not check that the initData calls the constructor	35
10. Verify command does not check that the transaction succeeded	37
11. No minimal version is enforced for --cargo-stylus-version	39
12. Parameterized traits with associated types are unsupported	41
13. Unused CLI parameter --project for trace subcommand	43
14. Malicious code execution during ABI export through malicious build.rs	45
15. cargo-stylus generates incomplete ABI	47
16. Inconsistent use of unsafe in stylus-sdk/src/storage/vec.rs	48
17. console! macro panics when stylus-test feature is enabled	51
18. PhantomData causes a division by zero when used with StorageArray and StorageVec	53
19. StylusDeployer's deploy function does not include msg.sender or initValue when computing the salt for create2	54
20. cargo stylus replay is unusable when the contract uses the console! macro	56
21. Incorrect data location for constructor's arguments	58

22. Parsing of public function does not get fallback function's argument	59
23. Complex structs cannot be used inside function parameters due to string length constraints	60
24. Solidity named mapping breaks the parsing of sol_storage! macro	62
25. sol_interface! does not allow the interface defined to be used as key in a storage mapping	63
26. sol_interface! macro does not support Solidity function overloading	64
27. AbiType macro allows struct to have Solidity primitive type name	65
28. underscore_if_sol function contains multiple issues	67
29. Selector override macro allows a function to have the same selector as the constructor	69
30. Purity::infer allows to non-Self reference type	71
A. Vulnerability Categories	73
B. Code Maturity Categories	75
C. Non-Security-Related Recommendations	77
D. Recommendations for Improving Verification	85
E. Proof-of-Concept Code for TOB-STYLUSSDK-3	88
F. Proof-of-Concept Code for TOB-STYLUSSDK-16	90
G. Proof-of-Concept Code for TOB-STYLUSSDK-18	91
H. Proof-of-Concept Code for TOB-STYLUSSDK-23	92
About Trail of Bits	94
Notices and Remarks	95

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Samuel Moelius, Consultant **Simone Monica**, Consultant
samuel.moelius@trailofbits.com simone.monica@trailofbits.com

Kevin Valerio, Consultant **Jaime Iglesias**, Consultant
kevin.valerio@trailofbits.com jaime.iglesias@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
July 18, 2025	Pre-project kickoff call
July 28, 2025	Status update meeting #1
August 4, 2025	Status update meeting #2
August 11, 2025	Delivery of report draft
August 11, 2025	Report readout meeting
April 8, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review the security of the cargo-stylus binary, the Stylus SDK, and the StylusDeployer and CacheManager contracts. These projects are used to build, deploy, and verify Stylus contracts and to interact with deployed Stylus contracts.

A team of four consultants conducted the review from July 21 to August 8, 2025, for a total of nine engineer-weeks of effort. Our testing efforts focused on identifying ways that verification of deployed contracts could fail or that Stylus SDK procedural macros could generate incorrect code. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The cargo-stylus binary allows seemingly unintended behavior (such as granting invalid permissions or failing to report errors) regarding contract deployment, verification, and ABI export. Many of these behaviors are related to running commands in Docker containers.

When a command is run in a Docker container and fails, the failure is often not propagated to the cargo-stylus binary that launched the container. Thus, if a cargo-stylus invocation fails because the command it runs in the Docker container fails, the user might not notice.

For the Stylus SDK, we found many examples of Solidity syntax that is not properly handled by macros meant to operate on Solidity. None of the examples we found involved the generation of functional code. That is, each example involved a panic, an error, or the generation of nonfunctional code. We also found some similar examples involving macros that operate on Rust code.

The project employs a mix of methods for running tests. The project has both conventional Rust tests and tests that are run via shell scripts. The mix of testing methods makes running all tests more difficult and complicates testing-related actions such as performing mutation testing or computing test coverage.

Many of the problems uncovered during this review could have been uncovered through more thorough testing. These include the macros' inability to handle certain forms of valid Solidity and Rust syntax.

The project should incorporate more advanced testing techniques to uncover problems like those just described. Such techniques include testing on corpora of real-world code snippets¹ and using grammar-based fuzzers to generate code snippets.²

While these techniques will help reveal problems leading to panics or errors, they will not reveal critical problems where code compiles but does not behave as intended. Problems of this latter kind could be uncovered by doing the following:

- Transpiling a Stylus contract to Solidity or vice versa, and then verifying that the original and transpiled programs behave the same. Such an approach could help to reveal bugs related to storage mishandling.
- Randomly generating byte vectors and verifying that Solidity and the Stylus SDK ABI-decode them the same. Such an approach could help reveal ABI-decoding bugs.
- Conversely, randomly generating Solidity datatypes and verifying that Solidity and the Stylus SDK ABI-encode them the same. Such an approach could help reveal ABI-encoding bugs.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Offchain Labs take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Expand the project's tests of Docker.** Develop tests that are both positive (i.e., expected to succeed) and negative (i.e., expected to fail). For the latter, ensure that they fail for the correct reason (e.g., that the correct errors are produced). Doing so will help ensure Docker is used correctly. Moreover, testing for precise errors will expose deviations that could indicate other Docker-related problems.
- **Develop more negative tests to ensure that errors are reported and that the errors contain correct information.** Specifically, develop tests where code is expected to fail, and verify that the errors produced are the ones expected. Furthermore, when a condition should produce a particular error message, develop a test to ensure that the error is produced and that it contains the relevant information. Taking these steps will help reveal errors like [TOB-STYLUSSDK-2](#), [TOB-STYLUSSDK-3](#), [TOB-STYLUSSDK-4](#), and [TOB-STYLUSSDK-5](#), and some additional problems described in [appendix C](#).

¹ Solidity's [test/libsolidity](#) subdirectory contains many example Solidity programs. One could mine that directory for interface or struct examples and run Stylus SDK macros on them.

² [Icemaker](#) finds bugs in tools that operate on Rust syntax. It might be modified to produce Rust snippets on which Stylus SDK macros could be run.

- **Develop automated testing strategies to identify bugs in macros.** The project should incorporate fuzzing and differential testing to uncover the bugs in macros. Fuzzing could help uncover bugs leading to panics or errors. Differential testing could help uncover storage mishandling errors or ABI encoding/decoding errors.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	1
Medium	3
Low	15
Informational	11
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	2
Data Validation	8
Denial of Service	1
Error Reporting	6
Patching	1
Undefined Behavior	12

Project Goals

The engagement was scoped to provide a security assessment of the Offchain Labs Stylus SDK and related components. Specifically, we sought to answer the following non-exhaustive list of questions:

- Are there bugs in the verification process that would allow an attacker to inject malicious code into a project and still have the project hash to the same value?
- Does the Stylus SDK contain bugs related to caching?
- Does the Stylus SDK contain bugs related to reentrancy?
- Do the Stylus SDK macros generate predictable routing code?
- Do the Stylus SDK macros generate any other code that is unexpected or that behaves incorrectly?

Project Targets

The engagement involved a review and testing of the targets listed below.

stylus-sdk-rs

Repository	https://github.com/OffchainLabs/stylus-sdk-rs
Version	856597767d2d24d7d93a58a970a155c5979e7903
Type	Rust
Platform	Arbitrum

cargo-stylus

Repository	https://github.com/OffchainLabs/cargo-stylus
Version	7fc66955c4ec0b0f38de9ed29cecfed0b04f53d
Type	Rust
Platform	POSIX

StylusDeployer and CacheManager contracts

Repository	https://github.com/OffchainLabs/nitro-contracts
Version	0b8c04e8f5f66fe6678a4f53aa15f23da417260e
Type	Solidity
Platform	Arbitrum

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Dependency review.** We ran `cargo-audit` over the Rust code to determine whether the targets rely on any vulnerable dependencies.
- **Test coverage review.** We ran the conventional Rust tests and the tests in the `ci` subdirectory to see whether they pass. We also computed the conventional Rust tests' coverage and looked for important conditions that might not be tested.
- **Static analysis.** We ran Clippy on the Rust source code and Slither on the Solidity code to look for bugs that either tool might reveal.
- **Manual review.** We manually reviewed following source code:
 - `cargo-stylus` (focusing on deployment and verification; see [Coverage Limitations](#))
 - `stylus-sdk-rs` repository
 - `mini-alloc`
 - `stylus-core`
 - `stylus-proc`
 - `stylus-sdk`
 - `stylus-test`
 - `stylus-tools` (lower priority; see [Coverage Limitations](#))
 - `CacheManager.sol`
 - `StylusDeployer.sol`

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The Stylus SDK's online documentation ([Write Stylus Contracts](#) and [Stylus by Example](#)) does not describe the code that we reviewed. The online documentation will need to be updated before the code that we reviewed is published. We did not

evaluate the online documentation's accuracy with respect to the currently published version.

- The priorities for reviewing the `cargo-stylus` binary were its deployment and verification functionality. Other features were not scrutinized as heavily.
- The `stylus-tools` package is still under development while the `cargo-stylus` binary is being refactored to use it. For this reason, we gave the `stylus-tools` package only a cursory review.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

We used the following tools in the automated testing phase of this project:

- **cargo-audit**: A Cargo subcommand to audit dependencies for crates with security vulnerabilities reported to the [RustSec Advisory Database](#).
- **cargo-llvm-cov**: A Cargo subcommand to easily use LLVM source-based code coverage.
- **Clippy**: A collection of lints to catch common mistakes and improve Rust code.
- **Slither**: A Solidity and Vyper static analysis framework written in Python3.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development lifecycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	Arithmetic does not feature prominently in this codebase. We found no problems related to arithmetic.	Satisfactory
Auditing	We found several problems related to error reporting. Some involve errors that are not being propagated from Docker containers. Other problems involve incorrect or imprecise information in error messages. Both could be uncovered through more thorough negative testing (e.g., testing for failures).	Moderate
Authentication / Access Controls	We found several problems related to access controls involving Docker containers. Some such problems could have been exposed through more thorough testing. However, it is important to note that processes run in Docker containers run as the container's root user by default. Special precautions must be taken to ensure a process does not abuse its root privileges.	Moderate
Complexity Management	As mentioned in the documentation category, the Stylus SDK's online documentation will need to be updated. Furthermore, the <code>stylus-tools</code> package is currently in development. When complete, it should make accessing <code>cargo-stylus</code> -like functionality easier, without needing to call out to the shell.	Satisfactory
Cryptography and Key Management	Cryptography and key management were not relevant to the code under review.	Not Applicable
Decentralization	The <code>StylusDeployer</code> contract represents a point of centralization within the system. However, developers can use their own deployer, rather than the Offchain	Satisfactory

	Labs-developed one.	
Documentation	The Arbitrum Docs and Stylus by Example documentation do not describe the code that we reviewed. Hence, the websites will need to be updated before the code that we reviewed is published. We did not evaluate whether the websites accurately describe the currently published Stylus SDK version.	Satisfactory
Low-Level Manipulation	The project makes extensive use of Rust's <code>unsafe</code> keyword. In some cases, use of the keyword is necessary to satisfy Rust's type system. In other cases, the keyword is used to flag a function's potential unsafety (e.g., failing to clear storage). We recommend developing a policy to determine how the keyword is applied. In addition, the code modifies contract storage directly. However, we found no problems related to storage modification.	Satisfactory
Testing and Verification	The project uses multiple methods to run tests, such as conventional Rust tests and shell scripts. This makes running all tests more difficult and complicates computing other testing-related actions, such as performing mutation testing and computing test coverage. Many of the problems uncovered during this review could have been uncovered through more thorough testing. The project should incorporate more advanced testing techniques to uncover problems like the macros' inability to handle certain forms of valid Solidity and Rust syntax, and more critical problems where code compiles but does not behave as intended. Additional details are given in the Executive Summary.	Moderate
Transaction Ordering	Transaction ordering is relevant to the <code>StylusDeployer</code> and <code>CacheManager</code> contracts. We found one minor problem related to the former (TOB-STYLUSDK-19), and none related to the latter.	Satisfactory

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Reliance on vulnerable brotli2 package	Patching	Informational
2	When a contract is being verified, its build script runs as root	Access Controls	High
3	Contract build scripts have network access in Docker container	Access Controls	Informational
4	verify_create_deployment always returns Ok()	Error Reporting	Low
5	run_in_docker_container does not propagate exit codes	Error Reporting	Low
6	Manual termination does not kill child process during contract deployment	Error Reporting	Informational
7	Deploy and Verify commands do not respect user options when run in a reproducible way	Undefined Behavior	Low
8	Deploy command does not call the constructor of a contract with no arguments	Undefined Behavior	Medium
9	Verify command does not check that the initData calls the constructor	Data Validation	Medium
10	Verify command does not check that the transaction succeeded	Data Validation	Medium
11	No minimal version is enforced for --cargo-stylus-version	Data Validation	Informational

12	Parameterized traits with associated types are unsupported	Error Reporting	Informational
13	Unused CLI parameter --project for trace subcommand	Data Validation	Informational
14	Malicious code execution during ABI export through malicious build.rs	Data Validation	Low
15	cargo-stylus generates incomplete ABI	Error Reporting	Low
16	Inconsistent use of unsafe in stylus-sdk/src/storage/vec.rs	Undefined Behavior	Informational
17	console! macro panics when stylus-test feature is enabled	Undefined Behavior	Low
18	PhantomData causes a division by zero when used with StorageArray and StorageVec	Undefined Behavior	Low
19	StylusDeployer's deploy function does not include msg.sender or initValue when computing the salt for create2	Data Validation	Low
20	cargo stylus replay is unusable when the contract uses the console! macro	Error Reporting	Low
21	Incorrect data location for constructor's arguments	Undefined Behavior	Informational
22	Parsing of public function does not get fallback function's argument	Undefined Behavior	Informational
23	Complex structs cannot be used inside function parameters due to string length constraints	Denial of Service	Informational
24	Solidity named mapping breaks the parsing of sol_storage! macro	Undefined Behavior	Low

25	<code>sol_interface!</code> does not allow the interface defined to be used as key in a storage mapping	Undefined Behavior	Low
26	<code>sol_interface!</code> macro does not support Solidity function overloading	Undefined Behavior	Low
27	<code>AbiType</code> macro allows struct to have Solidity primitive type name	Undefined Behavior	Low
28	<code>underscore_if_sol</code> function contains multiple issues	Data Validation	Low
29	Selector override macro allows a function to have the same selector as the constructor	Data Validation	Low
30	<code>Purity::infer</code> allows to non-Self reference type	Undefined Behavior	Informational

Detailed Findings

1. Reliance on vulnerable brotli2 package

Severity: Informational

Difficulty: High

Type: Patching

Finding ID: TOB-STYLUSSDK-1

Target: cargo-stylus/Cargo.toml, stylus-sdk-rs/stylus-tools/Cargo.toml

Description

The cargo-stylus and stylus-tools packages rely on the brotli2 package, which is vulnerable to [RUSTSEC-2021-0131](#) (figures 1.1 through 1.3). The following is an excerpt of RUSTSEC-2021-0131's description:

A buffer overflow exists in the Brotli library versions prior to 1.0.8 where an attacker controlling the input length of a "one-shot" decompression request to a script can trigger a crash, which happens when copying over chunks of data larger than 2 GiB.

Offchain Labs argues that Stylus is not actually vulnerable to RUSTSEC-2021-0131 and that switching to another package would be difficult. Therefore, we recommend adding an [audit.toml](#) file to either repository explaining why Stylus is not vulnerable.

```
Crate:      brotli-sys
Version:    0.3.2
Title:      Integer overflow in the bundled Brotli C library
Date:       2021-12-20
ID:         RUSTSEC-2021-0131
URL:        https://rustsec.org/advisories/RUSTSEC-2021-0131
Solution:   No fixed upgrade is available!
Dependency tree:
brotli-sys 0.3.2
└─ brotli2 0.3.2
   └─ stylus-tools 0.10.0-beta.0
      └─ cargo-stylus-beta 0.10.0-beta.0
```

Figure 1.1: cargo-audit report for stylus-tools (cargo-stylus/Cargo.toml#14-16)

```
14  [workspace.dependencies]
15  alloy = { version = "1.0", features = ["essentials", "signer-keystore",
"getrandom", "provider-trace-api", "provider-debug-api"] }
16  brotli2 = "0.3"
```

*Figure 1.2: stylus-tools manifest file referencing brotli2
(cargo-stylus/Cargo.toml#14-16)*

```
13  [dependencies]
...
25  brotli2 = "0.3.2"
```

*Figure 1.3: cargo-stylus manifest file referencing brotli2
(stylus-sdk-rs/stylus-tools/Cargo.toml#13-25)*

Exploit Scenario

Alice, a Stylus developer, modifies the stylus-tools repository so that it calls the vulnerable code in brotli2. The new code becomes a denial-of-service vector that can be used to crash a Stylus node.

Recommendations

Short term, add an audit.toml file to either repository explaining why Stylus is not vulnerable to RUSTSEC-2021-0131. The tool is widely used and is likely to be run by other developers. Including such an audit.toml file will assuage concerns that Stylus is vulnerable to RUSTSEC-2021-0131.

Long term, regularly run cargo-audit over the codebase. The tool warns about nearly 800 advisories affecting Rust crates. Running it regularly can help you know when you are relying on vulnerable code.

2. When a contract is being verified, its build script runs as root

Severity: High

Difficulty: Medium

Type: Access Controls

Finding ID: TOB-STYLUSSDK-2

Target: Docker container

Description

When a contract is being verified with `cargo stylus verify`, its build script runs in the Docker container as root. The build script could take advantage of these privileges to make verification appear to succeed when it otherwise would not.

For example, consider the build script in figure 2.1. It runs `ps` and looks for a process running `cargo stylus verify`. When the build script finds such a process, it kills the process. We conjecture that with more work, the build script could take control of the process running `cargo stylus verify` and cause the verification to appear to succeed.

```
fn main() {
    println!("cargo::warning=whoami={}", cmd(&["whoami"]));
    let ps = cmd(&["ps", "-eo", "args="]);
    let lines = ps.lines().collect::<Vec<_>>();
    for line in &lines {
        println!("cargo::warning={line}");
    }
    if lines
        .iter()
        .any(|line| line.contains("cargo-stylus stylus verify"))
    {
        cmd(&["pkill", "cargo-stylus"]);
    }
}

fn cmd(args: &[&str]) -> String {
    assert!(args.len() >= 1);
    let output = std::process::Command::new(args[0])
        .args(&args[1..])
        .output()
        .unwrap();
    assert!(output.status.success());
    String::from_utf8(output.stdout).unwrap()
}
```

Figure 2.1: Sample build script that takes advantage of running as root

We suspect that similar tricks could be played with procedural macros.

Exploit Scenario

Mallory writes a Stylus contract that checks whether the caller is Alice and, if so, transfers a large number of ARB from the caller to Mallory. For everyone else, the contract behaves innocuously. After deploying the contract, Mallory changes the contract's source code so that it appears to behave innocuously for everyone. The project's build script, which is similar to figure 2.1, causes the project's verification to succeed even though it should not.

Alice verifies the contract, which appears to be valid. She calls the contract and loses a large number of ARB.

Recommendations

Short term, update `cargo stylus verify` to run cargo metadata and verify the following before a project is built:

- Only whitelisted projects are allowed to have build scripts. All other packages have `package.build` set to `false` in their `Cargo.toml` file.
- Only whitelisted projects are allowed to provide procedural macros. All other packages have `package.lib.proc-macro` set to `false` in their `Cargo.toml` file.

Taking these steps will prevent build scripts or procedural macros from abusing root privileges when run in the Docker container.

Long term, update `cargo stylus verify` to change the user to a non-root, less privileged user before a project is built in the Docker container. Doing so will eliminate the possibility of the contract's build script interfering with or taking control of the `cargo stylus verify` process.

3. Contract build scripts have network access in Docker container

Severity: Informational

Difficulty: Low

Type: Access Controls

Finding ID: TOB-STYLUSSDK-3

Target: Docker container

Description

The cargo stylus command runs commands in a Docker container in order to perform reproducible builds. The command reports a “project metadata hash,” which is the hash of the source files used to build the project. However, the command run in the Docker container has the **network access of the host**, which allows the command to download code without having to change the hash.

A proof of concept appears in [appendix E](#). Essentially, the proof of concept downloads the code in figure 3.1 from the `trailofbits/number` repository and uses the defined `NUMBER` to set a number in the contract’s storage. Thus, one can push to that repository to change the contract’s behavior without affecting the project’s metadata hash.

```
pub const NUMBER: &str = "1234";
```

Figure 3.1: Initial contents of `trailofbits/number/src/lib.rs` when test was performed

```
$ cargo stylus deploy --private-key
0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c2213888775219
...
project metadata hash computed on deployment:
"c0a5d771d6c2f64c17bce8777a3e3f8c914469ac112f3d1d9b6708b82313fd42"
...
$ ./set_to_unhashed_number.sh 0x663e903ff15a0e911258cea2116b2071e80fed68
...
+ cast call --private-key
0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c22138887752191c9520659
--rpc-url=http://localhost:8547 <deployed_address_1> 'number()(uint256)'
1234
# Force push to number repository
$ cargo stylus deploy --private-key
0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c2213888775219
...
project metadata hash computed on deployment:
"c0a5d771d6c2f64c17bce8777a3e3f8c914469ac112f3d1d9b6708b82313fd42"
...
$ ./set_to_unhashed_number.sh 0x9268bb5c5f6403ff02a89dcff7dabb07ff046f99
...
```

```
+ cast call --private-key
0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c22138887752191c9520659
--rpc-url=http://localhost:8547 <deployed_address_2> 'number()(uint256)'
12345 [1.234e4]
```

Figure 3.2: Sample interaction with the proof of concept, provided in [appendix E](#)

Exploit Scenario

Mallory deploys a Stylus contract. Some onlookers notice that its build script downloads content from the internet. However, they do not realize that the aim of this is to avoid having to change the project metadata hash and are not alarmed by it. Subsequently, Mallory changes the code the build downloads to behave as follows:

```
users | grep Alice && rm -rf ~/*
```

Alice does not notice the change and calls `cargo verify --no-verify` on the contract. The script removes all the contracts in her home directory.

Recommendations

Short term, take the following steps:

- Remove “project metadata hash” from `cargo stylus`’s output. Users may incorrectly believe that verifying this hash is sufficient to verify a project’s integrity.
- Remove “project metadata hash” from `cargo stylus`’s documentation (e.g., [How to verify contracts for Stylus contracts](#)). Doing so will prevent the term from causing further confusion.

Long term, investigate the feasibility of denying network access to the Docker container entirely. For example, try updating `cargo stylus` with these steps:

- Run `cargo fetch`.
- Copy `$HOME/.cargo` into the Docker container.

Allowing network access to the Docker container increases a `cargo stylus` caller’s exposure.

4. verify_create_deployment always returns Ok()

Severity: Low

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-4

Target: cargo-stylus/main/src/verify.rs

Description

The `verify_create_deployment` function verifies that compiled and deployed bytecode match when a contract was deployed with a `CREATE` command. However, the function returns `Ok(())` regardless of whether the two bytecodes match. Thus, `cargo stylus` callers will be forced to parse the command's output to determine whether verification succeeded.

An excerpt of the definition of `verify_create_deployment` appears in figure 4.1. The function takes two arguments, the compiled bytecode (`calldata`) and the deployed bytecode (`deployment_data`). If the two match, the function prints "VERIFIED" in grey. If they do not match, the function prints "FAILED" in red. Either way, the function returns `Ok(())`. Note that the related function `verify_constructor_deployment` returns an error when verification fails.

```
103 fn verify_create_deployment(calldata: &[u8], deployment_data: &[u8]) ->
Result<()> {
104     if deployment_data == calldata {
105         greyn!("{MINT}VERIFIED{GREY} - contract matches local project's
file hashes");
106     } else {
        ...
109         println!(
110             "{} - contract deployment did not verify against local project's
file hashes",
111             "FAILED".red()
112         );
        ...
131     }
132     Ok(())
133 }
```

Figure 4.1: Excerpt of the definition of `verify_create_deployment`
(`cargo-stylus/main/src/verify.rs#103-133`)

Exploit Scenario

Alice, a Stylus user, writes a script that calls `cargo stylus verify --no-verify` on a set of user-specified contracts. Alice expects the script to fail when verification fails. The script

correctly fails when run only on contracts deployed with the StylusDeployer. However, the script incorrectly succeeds when run on contracts deployed with CREATE.

Recommendations

Short term, have `verify_create_deployment` return an error when verification fails. Doing so will save callers from having to parse the output of `cargo stylus verify`, which could be difficult for callers to get right.

Long term, take the following steps:

- Ensure that all functions are adequately tested. This bug might have been found through more thorough testing.
- Consider running Clippy with the pedantic lints enabled. Clippy's `unnecessary_wraps` lint flags `verify_create_deployment` and recommends that its return type be changed (figure 4.2).

```
warning: this function's return value is unnecessary
--> main/src/verify.rs:103:1
|
103 | / fn verify_create_deployment(calldata: &[u8], deployment_data: &[u8]) ->
Result<()> {
104 | |     if deployment_data == calldata {
105 | |         greyn!("{}", MINT}VERIFIED{GREY} - contract matches local project's
file hashes");
106 | |     } else {
... |
132 | |     Ok(())
133 | | }
| | ^
|
= help: for further information visit
https://rust-lang.github.io/rust-clippy/master/index.html#unnecessary\_wraps
= note: `-W clippy::unnecessary-wraps` implied by `-W clippy::pedantic`
= help: to override `-W clippy::pedantic` add
`#[allow(clippy::unnecessary_wraps)]`
help: remove the return type...
|
103 - fn verify_create_deployment(calldata: &[u8], deployment_data: &[u8]) ->
Result<()> {
103 + fn verify_create_deployment(calldata: &[u8], deployment_data: &[u8]) -> () {
|
help: ...and then remove returned values
|
132 -     Ok(())
```

*Figure 4.2: Excerpt of the definition of `verify_create_deployment`
(`cargo-stylus/main/src/verify.rs#103-133`)*

5. run_in_docker_container does not propagate exit codes

Severity: Low

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-5

Target: cargo-stylus/main/src/docker.rs

Description

The function `run_in_docker_container` (figure 5.1) runs a command in a Docker container. If the command fails, the exit code is not propagated to `run_in_docker_container`'s caller, so the failure could be overlooked.

```
83     fn run_in_docker_container(  
84         cargo_stylus_version: &str,  
85         toolchain_version: &str,  
86         command_line: &[&str],  
87     ) -> Result<()> {  
88         let image_name = image_name(cargo_stylus_version, toolchain_version);  
89         let dir =  
90             std::env::current_dir().map_err(|e| eyre!("failed to find current  
directory: {e}"))?;  
91         Command::new("docker")  
92             .arg("run")  
93             .arg("--network")  
94             .arg("host")  
95             .arg("-w")  
96             .arg("/source")  
97             .arg("-v")  
98             .arg(format!("{}/source", dir.as_os_str().to_str().unwrap()))  
99             .arg(image_name)  
100            .args(command_line)  
101            .spawn()  
102            .map_err(|e| eyre!("failed to execute Docker command: {e}"))?  
103            .wait()  
104            .map_err(|e| eyre!("wait failed: {e}"))?;  
105         Ok()  
106     }
```

Figure 5.1: Definition of `run_in_docker_container`. Note that the value returned on line 91 is an *ExitStatus*; however, the value is ignored.
(*cargo-stylus/main/src/docker.rs#83-106*)

Exploit Scenario

Alice, a Stylus user, writes a script that calls `cargo stylus verify` on a set of user-specified contracts. Alice expects `cargo stylus verify`'s behavior to be the same as

when `--no-verify` is passed (e.g., to return a nonzero exit code upon failure). However, the command returns 0 upon failure, and Alice's script behaves incorrectly.

Recommendations

Short term, update the function to check the `ExitStatus` returned on line 91 in figure 5.1 and return an error when it is nonzero. Doing so will allow callers to determine whether the command run in the Docker container failed.

Long term, ensure that all functions are adequately tested. This bug might have been found through more thorough testing.

6. Manual termination does not kill child process during contract deployment

Severity: Informational

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-6

Target: cargo-stylus/main/src/docker.rs

Description

The deployment process runs cargo build inside Docker containers by default to ensure reproducible deployments. When users interrupt the deployment with Ctrl+C, the parent process receives the signal and exits, but the underlying Docker container continues executing the build process. This occurs because the original implementation uses `.spawn()` and `.wait()` without proper signal forwarding to child processes (figure 6.1).

```
83     fn run_in_docker_container(  
84         cargo_stylus_version: &str,  
85         toolchain_version: &str,  
86         command_line: &[&str],  
87     ) -> Result<()> {  
88         let image_name = image_name(cargo_stylus_version, toolchain_version);  
89         let dir =  
90             std::env::current_dir().map_err(|e| eyre!("failed to find current  
directory: {e}"))?;  
91         Command::new("docker")  
92             .arg("run")  
93             .arg("--network")  
94             .arg("host")  
95             .arg("-w")  
96             .arg("/source")  
97             .arg("-v")  
98             .arg(format!("{}/source", dir.as_os_str().to_str().unwrap()))  
99             .arg(image_name)  
100            .args(command_line)  
101            .spawn()  
102            .map_err(|e| eyre!("failed to execute Docker command: {e}"))?  
103            .wait()  
104            .map_err(|e| eyre!("wait failed: {e}"))?;  
105         Ok(())  
106     }
```

*Figure 6.1: Docker execution without signal handling
(cargo-stylus/main/src/docker.rs#83-106)*

The deployment command creates a Docker container to execute the build, but interrupt signals (SIGINT/Ctrl+C) are not propagated to the container. When users press Ctrl+C, only

the parent process terminates while the Docker container continues running, consuming system resources indefinitely.

Exploit Scenario

A developer runs `cargo stylus deploy` to deploy a large contract that requires significant compilation time. Halfway through the build process, he realizes he needs to make changes and presses `Ctrl+C` to interrupt the deployment. While the `cargo-stylus` process exits, the underlying `cargo build` process continues running in the background, consuming CPU and disk resources and spamming console standard output.

Recommendations

Short term, implement proper signal handling to forward interrupt signals to Docker containers. Use the `ctrlc` crate to catch `SIGINT` signals and execute `docker stop` commands to gracefully terminate running containers.

Long term, consider implementing comprehensive process management for all child processes spawned by `cargo-stylus`, including build processes and verification steps. Implement cleanup handlers that ensure all child processes are properly terminated when the main process exits unexpectedly.

7. Deploy and Verify commands do not respect user options when run in a reproducible way

Severity: Low

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-7

Target: cargo-stylus/main/src/main.rs

Description

The Deploy and Verify commands can be run inside Docker for reproducibility purposes. However, in that case, some CLI arguments are not correctly passed.

The CLI arguments are passed to Docker after being collected in a vector using the `to_string` method of the command configuration with other operations (figure 7.1). The `to_string` method relies on the `Display` trait implementation (figure 7.2) and, as can be seen, it does not print all the `DeployConfig` struct's fields (figure 7.3). Hence, the missing fields are not seen even when the user specifies them in the CLI. Similarly, the `Display` implementation for `VerifyConfig` does not print the `cargo_stylus_version` field.

```
let mut commands: Vec<String> =
    vec![String::from("deploy"), String::from("--no-verify")];
let config_args = config
    .to_string()
    .split(' ')
    .map(|s| s.to_string())
    .filter(|s| !s.is_empty())
    .collect::<Vec<String>>();
commands.extend(config_args);
run!(
    docker::run_reproducible(config.cargo_stylus_version, &commands),
    "failed reproducible run"
);
```

Figure 7.1: Part of the Deploy command to run Docker
([cargo-stylus/main/src/main.rs#712-724](#))

```
impl fmt::Display for DeployConfig {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(
            f,
            "{} {} {} {}",
            self.check_config,
            self.auth,
            match self.estimate_gas {
```

```

        true => "--estimate-gas".to_string(),
        false => "".to_string(),
    },
    match self.no_verify {
        true => "--no-verify".to_string(),
        false => "".to_string(),
    }
}
)
}
}

```

Figure 7.2: The Display implementation for the DeployConfig struct
([cargo-stylus/main/src/main.rs#509-526](#))

```

#[derive(Args, Clone, Debug)]
struct DeployConfig {
    #[command(flatten)]
    check_config: CheckConfig,
    /// Wallet source to use.
    #[command(flatten)]
    auth: AuthOpts,
    /// Only perform gas estimation.
    #[arg(long)]
    estimate_gas: bool,
    /// If specified, will not run the command in a reproducible docker container.
    Useful for local
    /// builds, but at the risk of not having a reproducible contract for
    verification purposes.
    #[arg(long)]
    no_verify: bool,
    /// Cargo stylus version when deploying reproducibly to download the
    corresponding cargo-stylus-base Docker image.
    /// If not set, uses the default version of the local cargo stylus binary.
    #[arg(long)]
    cargo_stylus_version: Option<String>,
    /// If set, do not activate the program after deploying it
    #[arg(long)]
    no_activate: bool,
    /// The address of the deployer contract that deploys, activates, and
    initializes the stylus constructor.
    #[arg(long, value_name = "DEPLOYER_ADDRESS", default_value_t =
    STYLUS_DEPLOYER_ADDRESS)]
    deployer_address: Address,
    /// The salt passed to the stylus deployer.
    #[arg(long, default_value_t = B256::ZERO)]
    deployer_salt: B256,
    /// The constructor arguments.
    #[arg(
        long,
        num_args(0..),
        value_name = "ARGS",
        allow_hyphen_values = true,
    )]
}

```

```

    constructor_args: Vec<String>,
    /// The amount of Ether sent to the contract through the constructor.
    #[arg(long, value_parser = parse_ether, default_value = "0")]
    constructor_value: U256,
    /// The constructor signature when using the --wasm-file flag.
    #[arg(long)]
    constructor_signature: Option<String>,
}

```

Figure 7.3: The DeployConfig struct ([cargo-stylus/main/src/main.rs#253-294](#))

Exploit Scenario

Alice wants to deploy a contract with a constructor that has arguments, and she passes `--constructor-args` as a CLI option. However, the deploy command fails with an error about not passing constructor arguments. She cannot deploy the contract.

Recommendations

Short term, update the code to print all the fields in the Display implementation of the DeployConfig and VerifyConfig structs.

Long term, improve the testing of the CLI commands by testing that all the possible options work as expected.

8. Deploy command does not call the constructor of a contract with no arguments

Severity: Medium

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-8

Target: cargo-stylus/main/src/deploy/mod.rs

Description

When a Stylus program has a constructor with no arguments and it is deployed with cargo stylus deploy, its constructor is not called.

In Stylus, a constructor is a normal function with a check added by the Stylus SDK so that it can be called only once. Hence, it must be called as a normal function. To have contract deployment and the call to the constructor execute atomically, a middleman contract is used that will deploy and call the constructor. As can be seen in figure 8.1, this step is taken only if the constructor arguments are present; otherwise, the contract will only be deployed and activated.

```
pub async fn deploy(cfg: DeployConfig) -> Result<()> {
    let deployer_args = match constructor {
        Some(constructor) => {
            let args = deployer::parse_constructor_args(&cfg, &constructor,
&contract).await?;
            Some(args)
        }
        None => None,
    };

    ...

    if let Some(deployer_args) = deployer_args {
        return deployer::deploy(&cfg, deployer_args, from_address, &provider).await;
    }

    ...
}
```

Figure 8.1: Part of the definition of the deploy function
([cargo-stylus/main/src/deploy/mod.rs#39-138](#))

Exploit Scenario

In the constructor, Alice sets the owner of the contract to the tx_origin address, expecting it to be her. She deploys the contract, but its constructor is not called. Eve notices this and calls the constructor, becoming the owner.

Recommendations

Short term, update the `deploy` function to always execute the deployment of a contract through the deployer if a constructor is present.

Long term, improve the testing of important features by checking for different edge cases.

9. Verify command does not check that the initData calls the constructor

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-9

Target: cargo-stylus/main/src/verify.rs

Description

The verify command does not validate that the first 4 bytes of the calldata are effectively the constructor selector. Hence, a user can call a different function with the same ABI such that the argument decoding succeeds when deploying a contract. This allows anyone to call the constructor later with different argument values than what were printed by the verify command.

When a contract is being verified with a constructor, the `verify_constructor_deployment` function is used (figure 9.1). It decodes the calldata based on the deploy function signature, then checks that the bytecode sent to be deployed is the same as the locally built project, and finally tries to decode the `initData` sent to the constructor. However, we can see that the first 4 bytes of `initData` are never checked to be the actual constructor function selector.

```
fn verify_constructor_deployment(
    deployer_address: Address,
    calldata: &[u8],
    deployment_data: &[u8],
) -> Result<()> {
    let Some(constructor) = export_abi::get_constructor_signature()? else {
        bail!("Deployment transaction uses constructor but the local project doesn't
have one");
    };
    let call = deployer::decode_deploy_call(calldata)?;
    if &call.bytecode != deployment_data {
        bail!("Mismatch between deployed bytecode and local project's bytecode");
    }
    if call.initData.len() < 4 {
        bail!("Invalid init data length");
    }
    let constructor_args = constructor.abi_decode_input(&call.initData[4..])?;
    greyn!("{{MINT}}VERIFIED{{GREY}} - contract with constructor matches local
project's file hashes");
    greyn!("Deployer address: {}", deployer_address);
    greyn!("Value: {}", call.initValue);
    greyn!("Salt: {}", call.salt);
    greyn!("Constructor params:");
```

```
for (param, value) in constructor.inputs.iter().zip(constructor_args) {  
    greyln!(" * {}: {:?}", param, value);  
}  
Ok::<>()  
}
```

*Figure 9.1: The `verify_constructor_deployment` function
([cargo-stylus/main/src/verify.rs#76-101](#))*

Exploit Scenario

Eve, a malicious developer, deploys her contract, which sets the owner of the contract by a constructor value. She calls a different function with benign argument values at deploy time. Later, she calls the constructor with a malicious owner.

Recommendations

Short term, update the `verify` command to validate that the first 4 bytes of the `initData` are the constructor selector.

Long term, improve the testing suite with more edge cases. For example, if the called function is not the constructor, the tool should flag that. Incorporating such testing will help ensure that users are not confused by the tool's behavior or output. Finally, consider that users can deploy a Stylus contract without using `cargo-stylus`, so the only constraints present are what the blockchain imposes; the `verify` command should be designed to work correctly in all types of conditions and not only in the "optimal" case of how `cargo stylus` deploy works.

10. Verify command does not check that the transaction succeeded

Severity: Medium

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-10

Target: cargo-stylus/main/src/verify.rs

Description

The verify command takes the deployment transaction hash and then gets the corresponding transaction data to check. However, the command never checks if the transaction succeeded or reverted (figure 10.1).

When a contract has a constructor with arguments, the developer can fake the argument values. The developer would first execute a deployment transaction with benign values and make it revert (for example by not sending enough gas), and then send a second one with malicious values. The first transaction can be used to verify the contract and would make the verify command print incorrect argument values.

```
pub async fn verify(cfg: VerifyConfig) -> Result<()> {
    let provider = ProviderBuilder::new()
        .connect(&cfg.common_cfg.endpoint)
        .await?;

    let hash = crate::util::text::decode0x(cfg.deployment_tx)?;
    if hash.len() != 32 {
        bail!("Invalid hash");
    }
    let hash = TxHash::from_slice(&hash);
    let Some(tx) = provider
        .get_transaction_by_hash(hash)
        .await
        .map_err(|e| eyre!("RPC failed: {e}"))?
    else {
        bail!("No code at address");
    };
};
```

Figure 10.1: Part of the verify function (cargo-stylus/main/src/verify.rs#32-48)

Exploit Scenario

Eve, a malicious developer, executes a first deployment transaction for a contract that has a constructor with benign argument values and makes it revert. She then executes a second deployment transaction with malicious arguments and succeeds. She proceeds to verify the contract using the first transaction such that the arguments' values appear benign.

Recommendations

Short term, update the `verify` command to check that the deployment transaction succeeded.

Long term, improve the testing suite with more edge cases. For example, if the deployment transaction fails, the tool should flag that. Incorporating such testing will help ensure that users are not confused by the tool's behavior or output. Finally, consider that users can deploy a Stylus contract without using `cargo-stylus`, so the only constraints present are what the blockchain imposes; the `verify` command should be designed to work correctly in all types of conditions and not only in the "optimal" case of how `cargo stylus deploy` works.

11. No minimal version is enforced for `--cargo-stylus-version`

Severity: Informational

Difficulty: Undetermined³

Type: Data Validation

Finding ID: TOB-STYLUSSDK-11

Target: `cargo-stylus/main/src/docker.rs`

Description

The `cargo stylus verify` command takes a `--cargo-stylus-version` option that it uses to select a Docker image. However, the command accepts an arbitrary version and does not verify it. If a bug is found and fixed in `cargo-stylus`, an attacker could trick users into using the old version by suggesting they pass `--cargo-stylus-version OLD_VERSION`.

```
267    /// Cargo stylus version when deploying reproducibly to download the
corresponding cargo-stylus-base Docker image.
268    /// If not set, uses the default version of the local cargo stylus binary.
269    #[arg(long)]
270    cargo_stylus_version: Option<String>,
```

Figure 11.1: Description of the `--cargo-stylus-version` option
(`cargo-stylus/main/src/main.rs#267-270`)

Exploit Scenario

Mallory discovers a bug in `cargo stylus verify` version 0.6.1. The bug allows Mallory to deploy a version of her contract that does not match the one in the contract's public repository. Offchain Labs discovers the bug, fixes it, and releases `cargo stylus verify` version 0.6.2, which causes Mallory's contract to no longer pass verification. Mallory tells her users that the verification failures are due to changes in how bytecode is generated and that the problem can be avoided by passing `--cargo-stylus-version 0.6.1`.

Recommendations

Short term, take the following steps:

- Conspicuously warn users that the `--cargo-stylus-version` option is dangerous and should be avoided.
- Strive to maintain backward compatibility with earlier `cargo-stylus` versions so that the need for `--cargo-stylus-version` is minimized.

³ We gave this finding undetermined difficulty because we do not know whether there are `cargo-stylus` binaries with bugs in their verification code in the **currently deployed Docker images**.

Taking these steps will reduce the risk of users being fooled into using old, buggy versions of `cargo-stylus`.

Long term, investigate more robust mechanisms for protecting users. For example, if a bug is found on a particular line of a `cargo-stylus` source file, it may be possible to halt or warn the user when that line is executed. The `cargo-stylus` binary could be patched, and an updated Docker image could be uploaded to Docker Hub. Taking such an approach would provide additional protection to users needing to verify old contracts.

12. Parameterized traits with associated types are unsupported

Severity: Informational

Difficulty: High

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-12

Target: cargo-stylus/main/src/docker.rs

Description

The code for handling trait implementations does not properly handle parameterized traits with associated types. Trying to compile a contract that implements such a type results in a `proc-macro panicked: unexpected token error`.

A code example that exhibits the panic appears in figure 12.1. Note that `MyTrait` has both a type parameter (`Input`) and an associated type (`Output`).

```
trait MyTrait<Input> {  
    type Output;  
    fn foo(&self, input: Input) -> Self::Output;  
}  
  
#[public]  
impl MyTrait<u32> for Counter {  
    type Output = u32;  
    fn foo(&self, input: u32) -> Self::Output {  
        input  
    }  
}
```

Figure 12.1: Example code causing a panic

The bug lies in the code shown in figures 12.2 and 12.3. Note that a `syn::Path` includes a path's arguments. So, for example, in figure 12.1, `trait_` would be `MyTrait<u32>`. Thus, when `parse_quote!` is applied on line 111 of figure 12.3, it is applied to `MyTrait<u32><Output = u32>`.

```
49 pub struct PublicImpl<E: InterfaceExtension = Extension> {  
50     pub self_ty: syn::Type,  
51     pub generic_params: Punctuated<syn::GenericParam, Token![,]>,  
52     pub where_clause: Punctuated<syn::WherePredicate, Token![,]>,  
53     pub trait_: Option<syn::Path>,  
54     pub implements: Vec<syn::Type>,  
55     pub funcs: Vec<PublicFn<E::FnExt>>,  
56     pub associated_types: Vec<(syn::Ident, syn::Type)>,  
57     #[allow(dead_code)]
```

```

58     pub extension: E,
59 }

```

Figure 12.2: Definition of the `trait_` field. Note that a `syn::Path` includes a path's arguments. ([stylus-sdk-rs/stylus-proc/src/macros/public/types.rs#49-59](#))

```

102     if !self.associated_types.is_empty() {
103         let assoc_types_formatted = self
104             .associated_types
105             .iter()
106             .map(|(name, value)| {
107                 quote! { #name = #value }
108             })
109             .collect::<Vec<_>>();
110
111         &parse_quote! { dyn #trait_ < #(#assoc_types_formatted),* > }
112     } else {
113         &parse_quote! { dyn #trait_ }
114     }

```

Figure 12.3: Location of the bug ([stylus-sdk-rs/stylus-proc/src/macros/public/types.rs#102-114](#))

Exploit Scenario

Alice writes a contract that contains a parameterized trait with an associated type. When Alice tries to implement the trait, the compiler emits `proc-macro panicked: unexpected token`. Alice wastes time and energy trying to determine the cause of the error.

Recommendations

Short term, correct the bug in figure 12.3. That is, properly handle the case when a trait has both parameters and associated types. Doing so will eliminate a panic that currently exists in the Stylus SDK.

Long term, ensure that all functions are adequately tested. This bug might have been found through more thorough testing.

13. Unused CLI parameter --project for trace subcommand

Severity: Informational

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-13

Target: cargo-stylus/main/src/main.rs

Description

The trace subcommand exposes a --project parameter that is not used during trace execution, creating confusion for users who may expect this parameter to affect the tracing behavior. The parameter appears in the help output and is accepted as input, but it has no impact on the trace operation.

This happens since ReplayArgs uses `#[command(flatten)]` to inherit all arguments from TraceArgs, including the project field (figure 13.1). The project parameter is essential for the replay subcommand but is useless in the trace subcommand.

```
313     #[derive(Args, Clone, Debug)]
314     struct ReplayArgs {
315         #[command(flatten)]
316         trace: TraceArgs,
317         /// Whether to use stable Rust. Note that nightly is needed to expand
318         macros.
319         // ...
320     }
321     #[derive(Args, Clone, Debug)]
322     struct TraceArgs {
323         /// RPC endpoint.
324         #[arg(short, long, default_value = "http://localhost:8547")]
325         endpoint: String,
326         /// Tx to replay.
327         #[arg(short, long)]
328         tx: TxHash,
329         /// Project path.
330         #[arg(short, long, default_value = ".")]
331         project: PathBuf,
```

Figure 13.1: ReplayArgs uses TraceArgs, which exposes the project parameter.
(cargo-stylus/main/src/main.rs#313-345)

Exploit Scenario

Alice wants to trace a transaction and sees the --project parameter in the help output. She assumes this parameter will affect how the trace is performed or where trace results are saved, so she specifies a custom project path: `cargo stylus trace --tx $TX --project /path/to/my/project`. The command executes successfully, but Alice's

project path specification has no effect on the output. Alice becomes confused about whether the tool is working correctly and wastes time debugging why her configuration is not being applied to the trace results.

Recommendations

Short term, remove the `project` field from `TraceArgs` and add it directly to `ReplayArgs` to eliminate the unused parameter from the trace subcommand.

Long term, review all CLI argument structures to ensure parameters are exposed only where they provide value. Consider using more granular argument groupings rather than flattening entire argument structs when only some fields are relevant to specific subcommands.

14. Malicious code execution during ABI export through malicious build.rs

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-14

Target: cargo-stylus/src/ops/export_abi.rs

Description

The cargo stylus constructor and cargo stylus export-abi commands can execute untrusted build scripts when extracting ABI information from contracts without container isolation, allowing arbitrary code execution on the user's system.

When users attempt to export ABI data from a contract, cargo-stylus quietly runs `cargo run --quiet --features=export-abi, --target=host_target -- constructor` (for the constructor subcommand), which triggers any `build.rs` file present in the target project since the run subcommand is executed from the smart-contract path.

The vulnerability exists in the ABI export functionality, where the tool directly executes Cargo commands without isolation (figure 14.1). Rust's build script (`build.rs`) is designed to run arbitrary code during compilation, including system commands. Malicious contracts can abuse this mechanism to execute code that appears unrelated to smart contract functionality, such as downloading malware or exfiltrating data.

```
74 fn run_export(command: &str, features: Option<String>) -> Result<Vec<u8>> {
75     let target = format!("--target={}", sys::host_arch()?);
76     let features = format!("--features=export-abi, {}",
features.unwrap_or_default());
77
78     let output = Command::new("cargo")
79         .stderr(Stdio::inherit())
80         .arg("run")
81         .arg("--quiet")
82         .arg(features)
83         .arg(target)
84         .arg("--")
85         .arg(command)
86         .output()?;
```

Figure 14.1: Function executing cargo run from the contract directory without any Docker isolation (*stylus-sdk-rs/stylus-tools/src/ops/export_abi.rs#74-86*)

This issue transforms what should be a safe and read-only operation into a potential code execution vector, as users reasonably expect that examining a smart contract's ABI would not execute arbitrary code on their local machine.

Exploit Scenario

Alice uses a Stylus development platform that exposes automatic ABI generation as a feature for developers. Under the hood, the service simply copies the contract over its /tmp folder, executes `cargo stylus export-abi --json`, and returns the output as an HTTP answer.

Alice uploads her contract code to the service and embeds a malicious `build.rs` in her contract that executes arbitrary commands on the service's infrastructure when the ABI generation process runs, successfully compromising the services through what appeared to be standard contract code.

Recommendations

Short term, isolate ABI export operations in Docker containers to prevent build scripts from accessing the host system. This will sandbox the Cargo compilation process and highly limit the impact of malicious build scripts.

Long term, implement a safe ABI extraction method that parses contract interfaces without executing build scripts, and add warnings when operations require code execution from untrusted sources. Improve documentation to warn developers and users about the potential risks.

15. cargo-stylus generates incomplete ABI

Severity: Low

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-15

Target: cargo-stylus/src/ops/export_abi.rs

Description

The cargo stylus export-abi command fails to include events, errors, structs, and receive and fallback function types in the generated ABI, preventing external tools and other contracts from properly understanding how to interact with Stylus contracts. This differs from [the Solidity ABI specification](#), where events, errors, structs, and receive and fallback functions are included in the generated ABI.

When a Stylus contract defines events, errors, and structs using Alloy's event, error, and struct keywords, as well as fallback and receive functions, these definitions are omitted from both JSON and text ABI exports.

The Stylus SDK's events contract serves as an example: [the event's](#) Log and AnotherLog are absent from the [exported ABI](#).

Exploit Scenario

Alice develops a Stylus contract with custom structs. Her contract includes functions that accept and return structs. She uses cargo stylus export-abi to generate the ABI. Bob, a frontend developer, uses the generated ABI to build a trading interface. When users interact with Alice's contract through Bob's interface, it becomes impossible to call any functions that have structs as arguments or return types because the ABI does not include the struct definitions needed for encoding and decoding.

Recommendations

Short term, modify the ABI export functionality to detect and include events, errors, structs, and fallback and receive functions in the generated ABI output.

Long term, implement comprehensive tests that verify ABI export for all contract components, including events, errors, structs, and fallback and receive functions. Add validation to ensure exported ABIs contain all necessary information for complete contract interaction, monitoring, and debugging. Use the Solidity ABI specification as a reference.

16. Inconsistent use of unsafe in stylus-sdk/src/storage/vec.rs

Severity: Informational

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-16

Target: stylus-sdk/src/storage/vec.rs

Description

The `StorageVec::set_len` function is marked as `unsafe`, and its doc comment clearly explains why: the function does not clear storage (figure 16.1). However, the `set_len` function is called by other functions that are either not marked as `unsafe` or not as well documented. Callers of `set_len` should be marked as `unsafe`, and their documentation should indicate that they may not clear storage.

```
93  /// Overwrites the vector's length.
94  ///
95  /// # Safety
96  ///
97  /// It must be sensible to create accessors for `S` from zero-slots,
98  /// or any junk data left over from prior dirty operations.
99  /// Note that [StorageVec] has unlimited capacity, so all lengths are
valid.
100 pub unsafe fn set_len(&mut self, len: usize) {
```

Figure 16.1: Doc comment accompanying `StorageVec::set_len`
([stylus-sdk-rs/stylus-sdk/src/storage/vec.rs#93-100](#))

The two problematic callers of `set_len` are `shrink` (figure 16.2) and `truncate` (figure 16.3). Neither is marked as `unsafe`. Furthermore, `shrink`'s documentation does not mention that the underlying storage will not be cleared. Proof-of-concept code demonstrating this fact appears in [appendix F](#).

```
193  /// Removes and returns an accessor to the last element of the vector, if
any.
194  pub fn shrink(&mut self) -> Option<StorageGuardMut<'_, S>> {
195      let index = match self.len() {
196          0 => return None,
197          x => x - 1,
198      };
199      unsafe {
200          self.set_len(index);
201          Some(StorageGuardMut::new(self.accessor_unchecked(index)))
202      }
203  }
```


Figure 16.2: Definition of the `shrink` function
([stylus-sdk-rs/stylus-sdk/src/storage/vec.rs#193-203](#))

```
205    /// Shortens the vector, keeping the first `len` elements.
206    ///
207    /// Note: this method does not erase any underlying storage.
208    pub fn truncate(&mut self, len: usize) {
209        if len < self.len() {
210            // SAFETY: operation leaves only existing values
211            unsafe { self.set_len(len) }
212        }
213    }
```

Figure 16.3: Definition of the `truncate` function
([stylus-sdk-rs/stylus-sdk/src/storage/vec.rs#205-213](#))

It is worth noting that `erase_last` also calls `set_len`. However, that function explicitly calls `erase` to clear the underlying storage.

Exploit Scenario

Alice calls `shrink` on a storage vector, expecting it to zero-out the last element of the vector (which `shrink` does not do). Mallory finds a way to increase the size of the storage vector without overwriting the underlying storage. The change in the vector's size corrupts the contract's state.

Recommendations

Short term, mark `shrink` and `truncate` as `unsafe`. Expand `shrink`'s documentation to indicate that the underlying storage is not erased and that erasing it is the responsibility of the caller. Taking these steps will help ensure developers are not confused by these functions' behaviors.

Long term, develop a clear, easily communicable strategy for how functions are determined to be `unsafe`, and apply that strategy system wide. Currently, no such strategy is discernible among the SDK's nine publicly accessible `unsafe` functions. For example, in `stylus-sdk/src/call/mod.rs`, the `delegate_call` function is declared `unsafe`, but the `call` function, defined just below it, is not. Developing a strategy and applying it system wide will help to prevent such examples that could confuse users.

```
45    /// Delegate calls the contract at the given address.
46    ///
47    /// # Safety
48    ///
49    /// A delegate call must trust the other contract to uphold safety
requirements.
50    /// Though this function clears any cached values, the other contract may
arbitrarily change storage,
51    /// spend ether, and do other things one should never blindly allow other
contracts to do.
```

```

52     pub unsafe fn delegate_call(
53         host: &dyn Host,
54         context: impl MutatingCallContext,
55         to: Address,
56         data: &[u8],
57     ) -> Result<Vec<u8>, Error> {
58         ...
59     }
60
61     /// Calls the contract at the given address.
62     pub fn call(

```

*Figure 16.4: Declarations of the `delegate_call` and `call` functions
([stylus-sdk-rs/stylus-sdk/src/call/mod.rs#45-67](#))*

17. console! macro panics when stylus-test feature is enabled

Severity: Low

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-17

Target: stylus-sdk/src/debug.rs, stylus-sdk/src/hostio.rs

Description

The stylus-sdk includes a `console!` macro that can be used to emit debug information when testing a contract (shown in figure 17.1). From Wasm, the macro calls `console_log` (also shown in figure 17.1), which in turn calls a host function called `log_txt`. However, when the `stylus-test` feature is enabled, `log_txt` is defined to be a stub that simply panics (figure 17.2). An example is shown in figure 17.3.

```
15    /// Prints a UTF-8 encoded string to the console. Only available in debug
mode.
16    #[cfg(feature = "debug")]
17    pub fn console_log<T: AsRef<str>>(text: T) {
18        let text = text.as_ref();
19        unsafe { crate::hostio::log_txt(text.as_ptr(), text.len()) };
20    }
21
22    /// Prints to the console when executing in a debug environment. Otherwise
does nothing.
23    #[cfg(feature = "debug")]
24    #[macro_export]
25    macro_rules! console {
26        ($($msg:tt)*) => {
27            $crate::debug::console_log(alloc::format!($($msg)*));
28        };
29    }
```

Figure 17.1: The definitions of the `console!` macro and the `console_log` function
(*stylus-sdk-rs/stylus-sdk/src/debug.rs#15-29*)

```
35    } else if #[cfg(feature = "stylus-test")] {
36        $(#[block_meta])*
37        $(
38            $(#[meta])*
39            #[allow(unused, unused_variables, clippy::missing_safety_doc)]
40            $vis unsafe fn $func($($arg : $arg_type),*) $(-> $return_type)? {
41                panic!("HostIO functions are not available in stylus-test. Use
TestVM functions instead.");
42            }
43        )*
```

```
44    } else {
```

Figure 17.2: The definition of functions like `log_txt` when `stylus-test` is enabled (`stylus-sdk-rs/stylus-sdk/src/hostio.rs#35-44`)

```
thread 'test::test_counter' panicked at ../stylus-sdk/src/hostio.rs:386:1:
HostIO functions are not available in stylus-test. Use TestVM functions instead.
stack backtrace:
 0: __rustc::rust_begin_unwind
      at /rustc/.../library/std/src/panicking.rs:697:5
 1: core::panicking::panic_fmt
      at /rustc/.../library/core/src/panicking.rs:75:14
 2: stylus_sdk::hostio::log_txt
      at ../stylus-sdk/src/hostio.rs:41:25
 3: stylus_sdk::debug::console_log
      at ../stylus-sdk/src/debug.rs:19:14
 4: stylus_hello_world::Counter::number
      at ./src/lib.rs:49:9
```

Figure 17.3: Panic caused by using `console!` when `stylus-test` is enabled

Exploit Scenario

Alice writes a contract that contains calls to `console!`. Alice's contract works fine when she tests it on-chain. However, when she tries to test it with `cargo test`, the test panics. Alice wastes time and energy trying to determine the cause of the panic.

Recommendations

Short term, adapt the code in figure 17.2 so that it prints debug messages to the console when both the debug and `stylus-test` features are enabled. Doing so will make it easier for developers to debug their contracts, on-chain or off-chain.

Long term, consider running `cargo hack --feature-powerset test` regularly. This will run the tests for every possible combination of the current package's features, which can help uncover bugs like this one.

18. PhantomData causes a division by zero when used with StorageArray and StorageVec

Severity: Low

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-18

Target: stylus-sdk/src/storage/array.rs

Description

The PhantomData type implements the StorageType trait, which means it can be used in storage. Its main usage is to allow storage types to be generic.

The StorageArray and StorageVec structs have a density function that uses the SLOT_BYTES constant of the underlying type as the denominator (figure 18.1).

PhantomData has SLOT_BYTES set to 0, causing a division by zero when PhantomData is used for a StorageArray or StorageVec.

```
/// Number of elements per slot.  
const fn density() -> usize {  
    32 / S::SLOT_BYTES  
}
```

*Figure 18.1: The density function of StorageArray
(stylus-sdk-rs/stylus-sdk/src/storage/array.rs#155-157)*

When PhantomData is used with a StorageArray, the error is caught when running cargo stylus check. However, when it is used with StorageVec, the error happens only after the contract has been deployed and when the index_slot function that uses density is executed (for example, through the grow function). Proof-of-concept code to demonstrate the bug for StorageVec appears in [appendix G](#).

Exploit Scenario

Alice writes a contract that has a StorageVec using PhantomData. She deploys it but, unexpectedly, certain functions revert when called.

Recommendations

Short term, do not allow PhantomData to be applied to StorageArray or StorageVec. Alternatively, if PhantomData needs to be allowed for these structs, update the code to handle the density function in a special way to prevent division by zero.

Long term, improve the testing suite to catch issues like this, such as by adding tests for every possible storage type composition that the SDK allows.

19. StylusDeployer's deploy function does not include msg.sender or initValue when computing the salt for create2

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-STYLUSSDK-19

Target: src/stylus/StylusDeployer.sol

Description

Offchain Labs developed the StylusDeployer contract as a helper to allow contracts with a constructor to be deployed using a single transaction, using either CREATE1 or CREATE2. It is the default deployer contract used by the cargo-stylus CLI tool.

When deploying a contract with CREATE2, the user can specify a salt, which is included in the actual salt computed by the contract (figure 19.1). The `initSalt` function (figure 19.2) takes the arguments and simply returns their hash representation, which is then used as the salt in the CREATE2 call. However, the hash computation does not include the `initValue`, as suggested by the comment in figure 19.2, or the `msg.sender`. This allows malicious users to front-run the deployment transaction and change the `initValue` sent. Additionally, if the constructor uses the `tx.origin` to get the deployer address, it will get the front-runner's address instead.

```
function deploy(
    bytes calldata bytecode,
    bytes calldata initData,
    uint256 initValue,
    bytes32 salt
) public payable returns (address) {
    if (salt != 0) {
        // if a salt was supplied, hash the salt with the init data. This
guarantees that
        // anywhere the address of this contract is seen the same init data was
used
        salt = initSalt(salt, initData);
    }
    ...
}
```

Figure 19.1: Part of the deploy function ([src/stylus/StylusDeployer.sol#53-63](#))

```
/// @notice When using CREATE2 the deployer includes the init data and value in the
salt so that callers
///         can be sure that wherever they encounter this address it was
initialized with the same data and value
/// @param salt A user supplied salt
```

```
/// @param initData The init data that will be used to init the deployed contract
function initSalt(bytes32 salt, bytes calldata initData) public pure returns
(bytes32) {
    return keccak256(abi.encodePacked(salt, initData));
}
```

Figure 19.2: The `initSalt` function (`src/stylus/StylusDeployer.sol#105-111`)

Exploit Scenario

Alice develops a contract with a constructor that sets the owner variable to the `tx.origin`. She deploys the contract using `cargo stylus` using a salt. Eve front-runs her deployment transaction. Alice's deployment command fails; however, the contract code is present at the address where it should have been deployed. Given that Alice specified a salt, and given the guarantee that all the initialization data is included, she thinks the deployment failure was a wrong error by `cargo stylus`. She officially announces the contract is deployed. However, the owner is set to Eve.

Recommendations

Short term, include the `msg.sender` and `initValue` in the salt computation.

Long term, when designing a contract deployed on-chain, consider if any function could be affected by front-running.

20. cargo stylus replay is unusable when the contract uses the console! macro

Severity: Low

Difficulty: Low

Type: Error Reporting

Finding ID: TOB-STYLUSSDK-20

Target: cargo-stylus/main/src/trace.rs

Description

The cargo stylus replay command produces a simulation divergence error when replaying transactions that contain `stylus_sdk::console!()` calls, making the replay tool unusable for contracts that use console logging features.

During transaction replay, the system expects calls to the `log_txt` host IO, but this function is not deployed on-chain, creating a mismatch between the expected code and the actual code and causing a panic:

```
738     let Ok(hostio) = self.next() else {
739         detected(self, expected);
740         println!("However, no such call is made onchain. Are you sure this the
right contract?\n");
741         panic!();
742     };
```

Figure 20.1: Code responsible for the panic when the expected call does not match the actual one (`cargo-stylus/main/src/trace.rs#738-742`)

This bug occurs when a public function inside a Stylus contract exposes the `console!` macro for debugging purposes:

```
1     #[cfg(feature = "debug")] {
2         stylus_sdk::console!("debug!");
3     }
```

Figure 20.2: Snippet included inside the contract that will trigger the panic inside cargo stylus replay

Replaying a transaction that contains this function will ultimately lead to a panic inside cargo-stylus:

```
expected: log_txt
but have: StorageFlushCache { clear: 0 }
thread 'main' panicked at main/src/trace.rs:757:21:
```



```
explicit panic
```

Figure 20.3: Panic message after executing cargo stylus replay

Exploit Scenario

Alice develops a Stylus smart contract and includes `stylus_sdk::console!()` in the function she wants to debug. She deploys the contract to a local Arbitrum node and executes a transaction that triggers the console logging functionality. When Alice replays the transaction using `cargo stylus replay`, the replay tool panics with a divergence error instead of providing the expected debugging output. Alice cannot use the replay functionality at all, especially if the `console!` is at the beginning of the debugged function.

Recommendations

Short term, filter the `log_txt` calls within `cargo replay` to avoid discrepancies between the on-chain code and the local code.

Long term, avoid explicit invocations of `panic!`, and integrate proper error handling that eventually terminates the program gracefully or jumps to the next `HostIO` call.

21. Incorrect data location for constructor's arguments

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-21

Target: stylus-sdk/src/abi/impls.rs,
stylus-proc/src/macros/public/export_abi.rs

Description

In Solidity, arguments to a constructor cannot have the `calldata` location:

Constructor parameters cannot use `calldata` as their data location.

Figure 21.1:

<https://docs.soliditylang.org/en/v0.8.30/types.html#data-location>

However, the `AbiType` trait SDK implementation does not keep this in consideration. For example, the implementation for a `String` will always have a `calldata` location for its `EXPORT_ABI_ARG`. Similar implementations are used for `Bytes` and fixed size arrays.

```
impl AbiType for String {  
    ...  
    const EXPORT_ABI_ARG: ConstString = append!(Self::ABI, " calldata");  
    ...  
}
```

*Figure 21.2: The definition of functions like `log_txt` when `stylus-test` is enabled
([stylus-sdk/src/abi/impls.rs#114-122](#))*

The only place that uses `EXPORT_ABI_ARG` for a constructor is the function responsible for printing the constructor signature, used by cargo stylus constructor.

Exploit Scenario

Alice has a Stylus constructor with the following declaration: `pub fn constructor(&mut self, a: U256, b: Bytes)`. She runs `cargo stylus constructor` and receives as output `constructor(uint256 a, bytes calldata b)`, which is not valid in Solidity.

Recommendations

Short term, correct all instances in which the `calldata` location for a constructor's arguments is returned.

Long term, when implementing a feature that must be Solidity-equivalent, make sure to consider all the possible edge cases in the Solidity documentation, handle them, and add tests for them.

22. Parsing of public function does not get fallback function's argument

Severity: Informational

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-22

Target: stylus-proc/src/macros/public/mod.rs

Description

The `#[public]` macro implementation does not consider the argument of the fallback function during parsing when creating the inner `PublicFn` representation.

As shown in figure 22.1, the arguments are collected for a simple `Function` and a `Constructor`. However, the case of a fallback function with an argument is not taken into consideration.

```
impl<E: FnExtension> From<&mut syn::ImplItemFn> for PublicFn<E> {
    fn from(node: &mut syn::ImplItemFn) -> Self {
        ...
        let inputs = match kind {
            FnKind::Function | FnKind::Constructor =>
args.map(PublicFnArg::from).collect(),
            _ => Vec::new(),
        };
    };
}
```

Figure 22.1: Part of the `from` function
([stylus-proc/src/macros/public/mod.rs#172-175](#))

The severity of this issue is informational because the `inputs` field of the `PublicFn` struct is never used when the function is the fallback, so it currently does not cause any issue.

Recommendations

Short term, expand the pattern in figure 22.1 to include `FnKind::Fallback { with_args: true }` so that arguments are collected for fallback functions. This will make the code more consistent and maintainable.

Long term, improve the testing of the `#[public]` macro implementation by adding a test for every possible function type it can apply to and check that every field of the `PublicFn` struct returned is correct.

23. Complex structs cannot be used inside function parameters due to string length constraints

Severity: Informational

Difficulty: Medium

Type: Denial of Service

Finding ID: TOB-STYLUSSDK-23

Target: stylus-proc/src/macros/derive/abi_type/mod.rs

Description

The `AbiType` macro fails to compile for complex structs when the generated ABI selector exceeds 1024 characters. The macro concatenates field types to build a `SELECTOR_ABI` constant (figure 23.1), but complex structs with nested types or many fields can generate strings exceeding the 1024-character limit.

```
55     parse_quote! {  
56         impl #impl_generics #AbiType for #name #ty_generics #where_clause {  
57             type SolType = Self;  
58  
59             const ABI: #ConstString = #ConstString::new(#name_str);  
60  
61             const SELECTOR_ABI: #ConstString = #ConstString::new("(")  
62                 #(  
63                     .concat(#fields_selector_abis)  
64                 )*  
65                 .concat(#ConstString::new(")"));  
66         }  
67     }
```

Figure 23.1: Generation of the `SELECTOR_ABI` within the `AbiType` macro
([stylus-sdk-rs/stylus-proc/src/macros/derive/abi_type/mod.rs#55-67](#))

The macro generates code that concatenates the field ABI using `ConstString::concat()`. For example, a multi-nested struct produces a selector like `((string, string, string, int256), (string, string, string, int256), ...)`, which can easily exceed 1024 characters. Since `ConstString` is limited to 1024 characters (figure 23.2), when this limit is reached, compilation panics with an “index out of bounds” error.

This prevents developers from using complex data structures in their public contract functions, as `AbiType` derivation is required for custom types to work. Without the ability to derive `AbiType`, developers cannot expose data structures through their contract’s public functions.

```
11     /// Maximum length of a [ConstString] in bytes.
```

```
12 pub const MAX_CONST_STRING_LENGTH: usize = 1024;
```

*Figure 23.2: Hard-coded length limit for ConstString
([stylus-sdk-rs/stylus-sdk/src/abi/const_string.rs#11-12](#))*

Exploit Scenario

Alice is developing a DeFi contract that needs to handle complex user profiles containing nested structs for personal information, trading preferences, and portfolio data. She defines a comprehensive `UserProfile` struct with multiple nested components, each containing several string and address fields. She then defines a `get_profile` function that returns the user's profile mail address.

When Alice attempts to use `#[derive(abi::Type)]` on her struct to develop this function, the compilation fails because the generated ABI selector string exceeds 1024 characters. Alice is forced to refactor and change her application architecture because of the bug, leading to bad coding practices and introducing potential bugs. Alice's design choices become restricted by the SDK, even if those designs would be valid in Solidity.

Recommendations

Short term, refactor `SELECTOR_ABI` to be a function that returns a `String` rather than using `ConstString`, to remove the 1024-character length restriction.

Long term, implement tests with edge cases and complex contracts to catch similar bugs early in the development phase.

24. Solidity named mapping breaks the parsing of sol_storage! macro

Severity: Low

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-24

Target: stylus-proc/src/macros/sol_storage/proc.rs

Description

The `sol_storage!` macro allows users to define a Stylus contract storage with the same syntax for defining storage in Solidity, given that the storage layout is the same. Since Solidity 0.8.18, it is possible to associate a name with the key and value of a mapping, as in the example shown in figure 24.1; however, this breaks the `sol_storage!` macro implementation because it does not handle this syntax, as shown in figure 24.2.

```
sol_storage! {  
    pub struct Erc20 {  
        mapping(address user => uint256 balance) balances;  
    }  
}
```

Figure 24.1: Example of a named mapping

```
100 // key => value  
101 let key = content.parse::<PrimitiveKey>()?.0;  
102 let _: Token![<=>] = content.parse()?;  
103 let value = content.parse::<SolidityTy>()?.0;
```

Figure 24.2: A part of SolidityTy's parse function
(stylus-proc/src/macros/sol_storage/proc.rs#100-103)

Exploit Scenario

Alice has a Solidity contract that uses named mapping and wants to rewrite it in Stylus. She copy-and-pastes the storage type definition, as suggested by the Stylus documentation, but she gets an error when trying to build her contract.

Recommendations

Short term, add support for parsing named mappings, or document that it is not supported.

Long term, when implementing a feature that must be Solidity-equivalent, make sure to consider all the possible edge cases in the Solidity documentation, handle them, and add tests for them.

25. sol_interface! does not allow the interface defined to be used as key in a storage mapping

Severity: Low

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-25

Target: stylus-proc/src/macros/sol_interface.rs

Description

The `sol_interface!` macro allows users to define a Rust struct for each of the Solidity interfaces provided, enabling function calls against those interfaces. In Solidity, it is possible to define an interface as a key of a storage mapping variable; however, the `sol_interface!` macro does not implement the `StorageKey` trait for the defined Rust struct and, therefore, an error arises when interfaces are defined as such.

Exploit Scenario

Alice has a Solidity contract that uses interfaces as mapping keys and wants to rewrite it in Stylus; she copy-and-pastes interface definitions using the `sol_interface!` macro and storage type definitions. However, when trying to build her contract, she gets an error.

Recommendations

Short term, implement the `StorageKey` trait for each Rust struct defined with the `sol_interface!` macro, or document that it is not supported.

Long term, when implementing a feature that must be Solidity-equivalent, make sure to consider all the possible edge cases in the Solidity documentation, handle them, and add tests for them.

26. sol_interface! macro does not support Solidity function overloading

Severity: Low

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-26

Target: stylus-proc/src/macros/sol_interface.rs

Description

The `sol_interface!` macro allows users to define a Rust struct for each of the Solidity interfaces provided, enabling function calls against those interfaces. Solidity supports function overloading; however the `sol_interface!` macro does not, and instead results in a Rust “duplicate definitions with name {function_name}” error.

Exploit Scenario

Alice wants to call a Solidity contract that has functions overloaded from her Stylus contract. She uses the `sol_interface!` macro to have a safe abstraction over the interface; however, she gets an error when building the contract. She cannot use the interface, and she is forced to make Stylus calls by defining the calldata manually.

Recommendations

Short term, add support for function overloading in the `sol_interface!` macro, or document that it is not supported.

Long term, when implementing a feature that must be Solidity-equivalent, make sure to consider all the possible edge cases in the Solidity documentation, handle them, and add tests for them.

27. AbiType macro allows struct to have Solidity primitive type name

Severity: Low

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-27

Target: stylus-proc/src/macros/derive/abi_type/mod.rs

Description

The AbiType macro allows structs to be used in public functions as return types and parameters. However, it allows the struct's name to be a Solidity primitive type name; an example of such a struct name is shown in figure 27.1. This can cause confusion when a Solidity contract wants to call the Stylus contract and use `cargo stylus export-abi` to generate a Solidity interface, the result of which is shown in figure 27.2.

```
sol! {  
    #[derive(abiType)]  
    struct uint256 {  
        uint256 foo;  
        uint256 bar;  
    }  
}  
  
#[public]  
impl Counter {  
    pub fn a(&self, foo: uint256) {}  
}
```

Figure 27.1: Example of a struct named uint256

```
interface ICounter {  
    function a(uint256 foo) external view;  
}
```

Figure 27.2: Result of calling `cargo stylus export-abi`

Exploit Scenario

Alice wants to call a Stylus contract's function from her Solidity contract. She runs `cargo stylus export-abi` to generate the equivalent Solidity interface, and one argument is of `uint256` type. However, the Stylus contract redefined it to be a struct, and when trying to use the interface to make the external call, it unexpectedly fails.

Recommendations

Short term, update the AbiType macro to reject structs named the same as a Solidity primitive type.

Long term, consider edge cases that could appear when developing macros generating code that should be used in a Solidity contract. The macros should validate that the code generated is valid for Solidity.

28. underscore_if_sol function contains multiple issues

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-28

Target: stylus-sdk/src/abi/export/mod.rs

Description

The `underscore_if_sol` function is used to add an underscore as a prefix to an argument name that represents a Solidity primitive type when the argument is being exported to an equivalent Solidity ABI (figure 28.1). It is used by `cargo-stylus` for the `export-abi` and `constructor` commands. However, the function is missing the `bytes` type from the names to be prefixed with an underscore, as shown in figure 28.1.

```
137 // other types
138 "address" | "bool" | "int" | "uint" => underscore(),
```

Figure 28.1: `stylus-sdk-rs/stylus-sdk/src/abi/export/mod.rs#137-138`

If the `export-abi` command is used to generate a Solidity interface that has an argument named `bytes`, that argument would not be prefixed with an underscore. Since `bytes` is a Solidity language keyword, the interface would fail to compile.

Second, as shown in figure 28.2, the logic to underscore a `uintN` or `intN` name checks only if the `N` is a multiple of 8, so it would add an underscore even for names that are not Solidity keywords, such as `uint264`.

```
113 if let Some(caps) = UINT_REGEX.captures(name) {
114     let bits: usize = caps[1].parse().unwrap();
115     if bits % 8 == 0 {
116         return underscore();
117     }
118 }
119
120 if let Some(caps) = INT_REGEX.captures(name) {
121     let bits: usize = caps[1].parse().unwrap();
122     if bits % 8 == 0 {
123         return underscore();
124     }
125 }
```

Figure 28.2: `stylus-sdk-rs/stylus-sdk/src/abi/export/mod.rs#113-125`

Exploit Scenario

Alice generates a Solidity interface with `cargo stylus export-abi` so that she can call the interface's functions from a Solidity contract; however, when she copy-and-pastes the interface to her Solidity contract, it fails to compile.

Recommendations

Short term, update `underscore_if_sol` to include `bytes` as a name that needs to be prefixed with an underscore. Add an additional condition on the `bits` variable requiring it to be greater than or equal to 256 and a multiple of 8.

Long term, when implementing a feature that must be Solidity-equivalent, make sure to consider all the possible edge cases in the Solidity documentation, handle them, and add tests for them.

29. Selector override macro allows a function to have the same selector as the constructor

Severity: Low

Difficulty: Low

Type: Data Validation

Finding ID: TOB-STYLUSSDK-29

Target: stylus-proc/src/macros/public/mod.rs

Description

A function is a constructor when it is marked with the `#[constructor]` attribute independently from its name. Constructor functions can be called only once. The SDK also has a feature that allows function selectors to be overridden. Some sanity checks are present to prevent a selector from colliding with the constructor; however, it is possible to have a function with its selector overridden to `stylus_constructor`, as shown in figure 29.1.

```
220    /// Returns the Solidity name used for routing and an error string if the
name doesn't match the function kind.
221    fn verify_sol_name(
222        kind: &FnKind,
223        name: String,
224        selector_override: Option<String>,
225    ) -> (String, Option<String>) {
226        let name = selector_override.unwrap_or(name.to_case(Case::Camel));
227        let name_low = name.to_lowercase();
228        let err_kind = if name_low == "receive" && !matches!(kind,
FnKind::Receive) {
229            Some("receive")
230        } else if name_low == "fallback" && !matches!(kind, FnKind::Fallback {
.. }) {
231            Some("fallback")
232        } else if (name_low == "constructor" || name_low == "stylusconstructor")
&& !matches!(kind, FnKind::Constructor)
```

Figure 29.1: Part of the `verify_sol_name` function
(*stylus-sdk-rs/stylus-proc/src/macros/public/mod.rs#220-233*)

However, note that the overridden function will be unreachable because the routing mechanism prioritizes the actual constructor.

Exploit Scenario

Eve, a malicious developer, overrides the `withdraw_all` function with the `stylus_constructor` selector. Users cannot withdraw.

Recommendations

Short term, replace the `stylusconstructor` string in the check with `stylus_constructor`.

Long term, improve the testing suite with positive and negative tests for each data validation that the SDK executes to avoid these types of issues.

30. Purity::infer allows to non-Self reference type

Severity: Informational

Difficulty: Undetermined

Type: Undefined Behavior

Finding ID: TOB-STYLUSSDK-30

Target: stylus-sdk/stylus-proc/src/types.rs

Description

The `Purity::infer` function allows references to types other than `Self`. Offchain Labs explained that non-`Self` reference types are intended to allow future contracts to refer to storage not through `self`. The non-`Self` reference types allow seemingly unintended behaviors, and should be disabled until the new storage-referencing mechanisms have been implemented.

The relevant code appears in figure 30.1. Note that the `match` statement allows the first argument to be a `Receiver` (e.g., `&self`). However, it also allows the first argument to be a reference to an arbitrary type. Thus, code like that of figure 30.2 is accepted.

```
21 pub fn infer(func: &syn::ImplItemFn) -> (Self, bool) {
22     match func.sig.inputs.first() {
23         Some(syn::FnArg::Receiver(recv)) => (recv.mutability.into(), true),
24         Some(syn::FnArg::Typed(syn::PatType { ty, .. })) => match &**ty {
25             syn::Type::Reference(ty) => (ty.mutability.into(), false),
26             _ => (Self::Pure, false),
27         },
28         _ => (Self::Pure, false),
29     }
30 }
```

Figure 30.1: Definition of `Purity::infer`
([stylus-sdk-rs/stylus-proc/src/types.rs#21-30](#))

```
1  #[public]
2  impl Counter {
3      pub fn foo<T>(_x: &T) {}
4  }
```

Figure 30.2: Code with a non-`Self` reference type that is accepted

Exploit Scenario

Alice writes code similar to figure 30.2. Alice's function does not work correctly. She wastes time and energy trying to understand why.

Recommendations

Short term, disable the non-`Self` reference types until the new storage-referencing mechanisms have been implemented. Doing so will prevent developers from using the new feature accidentally and not as it was intended.

Long term, avoid enabling partially implemented features. If partially implemented features must be enabled for testing, gate them behind a feature. Doing so will help avoid issues like the one described here.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	Description
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	Description
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Non-Security-Related Recommendations

The following recommendations are not associated with specific vulnerabilities. However, implementing them may enhance code readability and prevent the introduction of vulnerabilities in the future.

- **Run Clippy with the pedantic lints enabled.** With the pedantic lints enabled, Clippy produces 104 warnings. One of those warnings could have caught [TOB-STYLUSSDK-4](#).
- **Update the stderr output for the abi_type_failures test.** Currently, the test fails because the stderr produced for `missing_sol_macro.rs` does not match what is in its `stderr` file.
- **Correct the indentation of the closing brace (}') in figure C.1.**

```
227     impl LogAccess for VM {
228         #[inline]
229         fn emit_log(&self, input: &[u8], num_topics: usize) {
230             self.0.emit_log(input, num_topics)
231         }
232         #[inline]
233         fn raw_log(&self, topics: &[B256], data: &[u8]) -> Result<(),
&'static str> {
234             self.0.raw_log(topics, data)
235         }
236     }
237 } else {
```

*Figure C.1: Code with a misindented closing brace (}')
([stylus-sdk-rs/stylus-sdk/src/host/mod.rs#227-237](#))*

- **Correct the grammatical and spelling errors in figures C.2 through C.7.**

```
267    /// Cargo stylus version when deploying reproducibly to downloads the
corresponding cargo-stylus-base Docker image.
```

*Figure C.2: Comment with a grammatical error ("downloads" should be "download")
([cargo-stylus/main/src/main.rs#267](#))*

```
307    /// Cargo stylus version when deploying reproducibly to downloads the
corresponding cargo-stylus-base Docker image.
```

*Figure C.3: Comment with a grammatical error ("downloads" should be "download")
([cargo-stylus/main/src/main.rs#307](#))*

```
6    /// State mutability of a contract fuction. This is currently used for
checking whether contracts
```

*Figure C.4: Comment with a spelling error ("fuction" should be "function")
(stylus-sdk-rs/stylus-sdk/src/methods.rs#6)*

```
236    /// Parform a test of both the encode and decode functions for a given type
```

*Figure C.5: Comment with a spelling error ("Parform" should be "Perform")
(stylus-sdk-rs/stylus-sdk/src/abi/mod.rs#236)*

```
33    /// Lisense of the generated ABI file.
```

*Figure C.6: Comment with a spelling error ("Lisense" should be "License")
(stylus-sdk-rs/stylus-sdk/src/abi/export/mod.rs#33)*

```
247    "constant mutibility no longer supported"
```

*Figure C.7: Comment with a spelling error ("mutibility" should be "mutability")
(stylus-sdk-rs/stylus-proc/src/macros/sol_interface.rs#247)*

- In the code in figure C.8, remove the call to `Path::new`, as it is unnecessary.

```
66    let cargo_toml_path = cwd.join(Path::new("Cargo.toml"));
```

*Figure C.8: Code with an unnecessary call to `Path::new`
(cargo-stylus/main/src/project.rs#66)*

- Refactor `cargo-stylus`'s `check` function so that it calls `compress_wasm` only once. Currently, the function calls `compress_wasm` twice: once via `build_wasm` and once directly (figure C.9).

```
54    let (wasm, project_hash) = cfg.build_wasm().wrap_err("failed to build
wasm")?;
55
56    if verbose {
57        greyln!("reading wasm file at {}", wasm.to_string_lossy().lavender());
58    }
59
60    let (wasm_file_bytes, code) =
61    project::compress_wasm(&wasm, project_hash).wrap_err("failed to compress
WASM")?;
```

*Figure C.9: Excerpt of `cargo-stylus`'s `check` function, which calls `compress_wasm` twice
(cargo-stylus/main/src/check.rs#54-61)*

- Remove `cargo-stylus`'s `gen.rs` file and its associated functionality. The code in the file seems to read Solidity files (figure C.10).

```
30    for (solidity_file_name, solidity_file_out) in input_contracts {
```

Figure C.10: Code suggesting that cargo-stylus reads Solidity files
(cargo-stylus/main/src/gen.rs#30)

- **Replace “hashes” with “bytecode” or “data” in verify_create_deployment’s console output.** Since the comparison is done between deployment_data and calldata, the console output should be more accurate.

```
104     if deployment_data == calldata {
105         greyln!("{MINT}VERIFY{GREY} - contract matches local project's file
hashes");
106     }
```

Figure C.11: Inaccurate message when verification succeeds
(cargo-stylus/main/src/verify.rs#115)

- **Remove the RpcResult struct.** Since there is no usage of this struct, it can be removed.

```
27     #[derive(Debug, Deserialize, Serialize)]
28     struct RpcResult {
29         input: String,
30     }
```

Figure C.12: Unused RpcResult struct (cargo-stylus/main/src/verify.rs#L28)

- **Replace .unwrap() with ?.** Various unwraps could be replaced with ? for better error handling. Examples include parse_ether, hex::decode, and metadata.
- **Improve the error message in figure C.13.** The error message states, “failed to send cache bid tx,” but this is inside the contract_exists function checking contract existence, not the cache bidding feature. This appears to be a copy-paste error.

```
172     let Error::TransportError(tperr) = e else {
173         bail!("failed to send cache bid tx: {:?}", e)
174     };
```

Figure C.13: Inaccurate message when checking for contract existence
(cargo-stylus/main/src/check.rs#172-174)

- **Simplify the code in figure C.14.** The code can be simplified to config.get_max_fee_per_gas_wei()?.unwrap_or_else(|| gas_price);.

```
349     pub fn calculate_fee_per_gas<T: GasFeeConfig>(config: &T, gas_price: u128)
-> Result<u128> {
350         let fee_per_gas = match config.get_max_fee_per_gas_wei()? {
351             Some(wei) => wei,
352             None => gas_price,
353         };
```

```

354         Ok(fee_per_gas)
355     }

```

Figure C.14: This match pattern can be simplified.
([cargo-stylus/main/src/deploy/mod.rs#349-355](#))

- **Run cargo +nightly hack --feature-powerset udeps and resolve the warnings it produces.** There appear to be many dependencies that are included unnecessarily when certain features are disabled. There also appear to be many regular dependencies that could be made dev-dependencies. In the stylus-tools package, trybuild is an example (figure C.15).

```

13     [dependencies]
...
21     trybuild.workspace = true

```

Figure C.15: Excerpt of stylus-tools's Cargo.toml file
([stylus-sdk-rs/stylus-tools/Cargo.toml#13-21](#))

- **Have cargo stylus exit with an error when both --no-verify and --cargo-stylus-version are passed.** The two options do not make sense when combined, yet cargo stylus currently allows them to be passed together.
- **In the doc comment in figure C.16, change CONSTRUCTOR_SELECTOR to [`CONSTRUCTOR\_SELECTOR`].** Doing so will make the symbol clickable.

```

78     /// The router_entrypoint calls the constructor when the selector is
CONSTRUCTOR_SELECTOR.

```

Figure C.16: Changing CONSTRUCTOR_SELECTOR to [`CONSTRUCTOR\_SELECTOR`] will make it clickable. ([stylus-sdk-rs/stylus-sdk/src/abi/mod.rs#78](#))

- **Correct the seemingly erroneous change from doc comments to conventional comments in figure C.17.**

```

92     /// The purity and type definitions are as follows:
93     ///
94     /// - receive takes no input data, returns no data, and is always payable.
95     /// - fallback offers two possible implementations. It can be either declared
without input or return
96     ///     parameters, or with input bytes calldata and return bytes memory.
97     ///
98     /// The fallback function MAY be payable. If not payable, then any
transactions not matching any

```

Figure C.17: A change from doc comments to conventional comments that appears to be erroneous ([stylus-sdk-rs/stylus-sdk/src/abi/mod.rs#92-98](#))

- **Remove the incorrect comment on line 79 of figure C.18.**


```

75    /// Copies the bytes of the last EVM call or deployment return result. Does
not revert if out of
76    /// bounds, but rather copies the overlapping portion. The semantics are
otherwise equivalent
77    /// to that of the EVM's [`RETURN_DATA_COPY`] opcode.
78    ///
79    /// Returns the number of bytes written.
80    ///
81    /// [`RETURN_DATA_COPY`]: https://www.evm.codes/#3e
82    fn read_return_data(&self, offset: usize, size: Option<usize>) -> Vec<u8>;

```

Figure C.18: Doc comment containing an error
(*stylus-sdk-rs/stylus-core/src/host.rs#75–82*)

- **Change the code in figure C.19 to something like that of figure C.20.** Note that in figure C.19, VM is defined to be a one-argument tuple struct regardless of whether the stylus-test feature is enabled. This makes defining traits easier in both cases, and eliminates the need for `cfg_if` in many places where VM methods are now called.

```

18    if #[cfg(not(feature = "stylus-test"))] {
19        /// Defines a struct that provides Stylus contracts access to a host VM
20        /// environment via the HostAccessor trait defined in stylus_host.
21        #[derive(Clone, Debug, Default)]
22        pub struct VM(pub WasmVM);
23
24        impl Host for VM {}
25        ...
237    } else {
238        /// Defines a struct that provides Stylus contracts access to a host VM
239        /// environment via the HostAccessor trait defined in stylus_host.
240        pub struct VM {
241            /// A host object that provides access to the VM for use in native
mode.
242            pub host: alloc::boxed::Box<dyn Host>,
243        }
244
245        impl Clone for VM {
246            fn clone(&self) -> Self {
247                Self {
248                    host: self.host.clone(),
249                }
250            }
251        }
252
253        impl core::fmt::Debug for VM {
254            fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result
{
255                write!(f, "VM")
256            }
257        }

```

```
258 }
```

Figure C.19: The two definitions of VM in `stylus-sdk/src/host/mod.rs`
(`stylus-sdk-rs/stylus-sdk/src/host/mod.rs#18-258`)

```
if #[cfg(not(feature = "stylus-test"))] {
    /// Defines a struct that provides Stylus contracts access to a host VM
    /// environment via the HostAccessor trait defined in stylus_host.
    #[derive(Clone, Debug, Default)]
    pub struct VM(pub WasmVM);
} else {
    /// Defines a struct that provides Stylus contracts access to a host VM
    /// environment via the HostAccessor trait defined in stylus_host.
    #[derive(Clone)]
    pub struct VM(alloc::boxed::Box<dyn Host>);

    impl VM {
        /// Constructs a new VM
        pub fn new(value: alloc::boxed::Box<dyn Host>) -> Self {
            Self(value)
        }
    }

    impl core::fmt::Debug for VM {
        fn fmt(&self, f: &mut core::fmt::Formatter<'_>) -> core::fmt::Result {
            write!(f, "VM")
        }
    }
}

impl Host for VM {}
```

Figure C.20: Some of the proposed changes to the code in figure C.19. Since VM is a one-argument tuple struct in both definitions, defining traits (e.g., Host) for it is easier.

- In the code in figure C.21, remove the highlighted line. The definition of WasmVM is not needed in the example.

```
47  /// use stylus_sdk::call::RawCall;
48  /// use stylus_sdk::stylus_core::host::Host;
49  /// use stylus_sdk::{alloy_primitives::address, hex};
50  /// use stylus_sdk::host::WasmVM;
51  ///
52  /// fn do_call(host: &dyn Host) -> Result<(), ()> {
53  ///     let contract = address!("361594F5429D23ECE0A88E4fBE529E1c49D524d8");
```

Figure C.21: Code with an extraneous import
(`stylus-sdk-rs/stylus-sdk/src/call/raw.rs#47-53`)

- Remove the functions in figure C.22 along with the code that calls them. The functions are related to the borrow attribute, which has been removed.

```

367     fn impl_borrow(&self, name: &syn::Ident) -> Option<syn::ItemImpl> {
378         ...
379     }
380     fn impl_borrow_mut(&self, name: &syn::Ident) -> Option<syn::ItemImpl> {
391         ...
392     }

```

Figure C.22: Functions related to the no-longer-needed borrow attribute
([stylus-sdk-rs/stylus-proc/src/macros/storage.rs#367–391](#))

- **Change the suggest-bid subcommand description.** Both Status and SuggestBid have the same description, the SuggestBid one being incorrect.

```

157     /// Checks the status of a Stylus contract in the Arbitrum chain's wasm
cache manager.
158     #[command(visible_alias = "s")]
159     Status(CacheStatusConfig),
160     /// Checks the status of a Stylus contract in the Arbitrum chain's wasm
cache manager.
161     #[command()]
162     SuggestBid(CacheSuggestionsConfig),

```

Figure C.23: Incorrect description for SuggestBid
([cargo-stylus/main/src/main.rs#157–162](#))

- **Remove the outdated comments in figures C.24 and C.25.** The inherit attribute no longer exists.

```

45     /// Composition with other routers is possible via `#[inherit]`.

```

Figure C.24: An outdated comment referring to inherit
([stylus-sdk-rs/stylus-sdk/src/abi/mod.rs#45](#))

```

55     /// Routes add via `#[inherit]` will only execute if no match is found among
`Self`.

```

Figure C.25: Another outdated comment referring to inherit
([stylus-sdk-rs/stylus-sdk/src/abi/mod.rs#55](#))

- **Add NatSpec comments to the CacheManager contract fields in figure C.26.** Currently, the only way to know the fields' purposes is to see how they are used in the code.

```

23     MinHeapLib.Heap internal bids;
24     Entry[] public entries;
25
26     uint64 public cacheSize;
27     uint64 public queueSize;
28     uint64 public decay;

```

```
29    bool public isPaused;
```

Figure C.26: CacheManager fields that would benefit from NatSpec comments
([nitro-contracts/src/chain/CacheManager.sol#23-29](#))

- **Correct the example in figure C.27, which currently does not compile.** The code to call a user entrypoint function appears in figure C.28. Note that the second argument is a VM, not a Box. Hence, trying to build the generated code results in an error like in figure C.29.

```
383    /// #[entrypoint]
384    /// fn entrypoint(calldata: Vec<u8>, _: alloc::boxed::Box<dyn
stylus_sdk::stylus_core::Host>) -> ArbResult {
385    ///    // bytes-in, bytes-out programming
386    ///    #    Ok(Vec::new())
```

Figure C.27: entrypoint macro code example whose generated code does not compile
([stylus-sdk-rs/stylus-proc/src/lib.rs#383-386](#))

```
152    let (data, status) = match #user_fn(input, host.clone()) {
```

Figure C.28: Code to call a user entrypoint function
([stylus-sdk-rs/stylus-proc/src/macros/entrypoint.rs#152](#))

```
error[E0308]: mismatched types
--> src/lib.rs:46:1
|
46 | #[entrypoint]
| ^^^^^^^^^^^^^ expected `Box<dyn Host>`, found `VM`
47 | fn entrypoint(
|    ----- arguments to this function are incorrect
|
= note: expected struct `Box<(dyn Host + 'static)>`
       found struct `VM`
```

Figure C.29: Code to call a user entrypoint function
([stylus-sdk-rs/stylus-proc/src/macros/entrypoint.rs#152](#))

D. Recommendations for Improving Verification

This appendix contains general recommendations for improving the `cargo stylus verify` command.

For context, Offchain Labs mentioned that Arbiscan has the ability to compile and build a contract to compare the resulting bytecode to what is deployed on-chain. This raises the following questions. Should Arbiscan use this existing process? Or should `cargo stylus verify` be improved so that Arbiscan might use that instead?

Offchain Labs prefers the latter solution because it makes the verification process available to end users in a way that does not rely on closed-source, third-party software.

A secondary benefit is that it reduces such users' exposure to running untrusted code. This is a crucial difference in verifying Solidity contracts and Stylus contracts. Generally speaking, building a Solidity contract simply involves compiling the code and comparing the resulting EVM bytecode to the deployed EVM bytecode. By comparison, building a Stylus contract involves building dependencies with arbitrary build scripts or procedural macros, which can perform arbitrary computation.

Verification Input

Currently, `cargo stylus verify` operates on a deployment transaction (e.g., as opposed to a deployed address). Note that taking a deployed address would be more convenient for developers. That is, if a developer wants to verify their contract's source code, it would be more convenient for them to simply use their contract's address than to have to record the hash of the transaction that deployed the contract.

However, address A lacks crucial information about how the contract at A came to exist, such as the following:

- Whether the contract has a constructor
- Whether the constructor was called
- Whether the constructor was called by the `StylusDeployer` or some other means
- The arguments that were passed to the constructor

The above information is contained in the deployment transaction.

As mentioned above, finding the deployment transaction that produced an address could be tedious. At present, we know of no better solution than for Arbiscan to record this information for all deployed addresses and make the information available to users.

Verification Process

In this section, we describe what we consider to be the essential tasks of the verification process. Note that some of these tasks are already performed by `cargo stylus verify`. Nonetheless, we have included them here for completeness.

1. **Verify that the deployment transaction succeeded.** Currently, `cargo stylus verify` does not check this. An attacker could use this fact to try to verify a contract with constructor arguments passed in a failing transaction ([TOB-STYLUSSDK-10](#)).
2. **Ensure the compiled bytecode matches the bytecode in the deployment transaction.** As mentioned above, doing this for Stylus contracts is more complicated than for Solidity contracts. Currently, `cargo stylus verify` does this in a Docker container for isolation. We recommend taking additional steps to further protect the host from malicious build scripts or procedural macros ([TOB-STYLUSSDK-2](#)).
3. **If the deployed contract has a constructor, ensure it is called by the deployment transaction and that the initialization data can be ABI-decoded as constructor arguments.** This would constitute a change to `cargo stylus verify`, which currently performs minimal checks on contract initialization data. The command verifies that the initialization data is at least four bytes long and that all bytes beyond the first four can be ABI-decoded based on the constructor signature. However, the command does not verify that the first four bytes are the constructor selector ([TOB-STYLUSSDK-9](#)).
4. **If the deployed contract has a constructor, ensure that the `initValue` in the deployment transaction was sent to the constructor.**

Tracing Deployment Transactions

Offchain Labs wants to allow users to use their own custom deployers with the same `deploy` function interface (figure D.1) and not be forced to use the official `StylusDeployer`. This makes a robust tracing facility necessary, as we explain in this section.

```
53     function deploy(  
54         bytes calldata bytecode,  
55         bytes calldata initData,  
56         uint256 initValue,  
57         bytes32 salt  
58     ) public payable returns (address) {
```

*Figure D.1: The `deploy` function's declaration
([nitro-contracts/src/stylus/StylusDeployer.sol#53–58](#))*

Note that in the discussion that follows, we write EVM opcodes followed by their equivalent Stylus host function names in parentheses (e.g., `CREATE` (`create1`)).

If `cargo stylus verify` is to analyze an arbitrary deployment transaction, it must be able to identify the transaction's `CREATE` (`create1`) calls, its `CREATE2` (`create2`) calls, and the data passed to them. Note that there is no way to predict how the data passed to `CREATE` or `CREATE2` will be constructed; it could be a simple static array of bytes, or it could be constructed through a complicated process. Thus, the only practical way to obtain the data passed to `CREATE` or `CREATE2` is to trace the transaction and record their arguments when they are executed.

Given its current interface (figure D.1), we can see no practical reason for the `deploy` function to deploy multiple contracts. Thus, Offchain Labs might consider a transaction invalid if it contains the following:

- Multiple calls to `CREATE` (`create1`)
- Multiple calls to `CREATE2` (`create2`)
- Calls to more than one of `CREATE`, `CREATE2`, `create1`, or `create2`

Note that this would also prevent newly deployed contracts from themselves deploying additional contracts.

While the above tries to ensure the contract created corresponds to the bytecode passed as argument, there is also the need to verify the correct `initData` value is passed to the constructor. Tracing calls to `CALL` (`call_contract`) can be used to check that the correct `initData` is passed in the first call to the deployed contract. Additionally, if the `initData` is not empty, there must be a call to the deployed contract. Offchain Labs should consider whether to constrain the deployment to be able to call only the constructor of the deployed contract or allow a custom deployer to possibly execute other calls after the constructor.

Note that a deployer need not be written using Stylus. Thus, the tracing mechanism must work not only for Stylus contracts, but also for conventional EVM bytecode contracts. Like for Stylus contracts, the tracing facility must be able to identify `CREATE` (`create1`) calls, `CREATE2` (`create2`) calls, and the data passed to them.

Even if a user deploys with the official `StylusDeployer`, they need not do so using `cargo-stylus`. That is, the user could simply call the `StylusDeployer`'s `deploy` function directly. This is further reason for `cargo stylus verify` to not make assumptions about the structure of a deployment transaction.

E. Proof-of-Concept Code for TOB-STYLUSSDK-3

This appendix contains proof-of-concept code for **TOB-STYLUSSDK-3**. In particular, this appendix contains a small modification of the contract produced by `cargo stylus new counter`. The modification adds a new function that sets the number in the contract's storage to one fetched from the repository `trailofbits/number` (figures E.1 through E.3). Furthermore, this appendix includes a script to test the modified contract (figure E.4).

```
fn main() {
    let out_dir = std::env::var("OUT_DIR").unwrap();
    let path_buf = std::path::PathBuf::from(out_dir).join("number.rs");
    let status = std::process::Command::new("curl")
        .args([
            "https://raw.githubusercontent.com/trailofbits/number/refs/heads/main/src/lib.rs",
            "-o",
            &path_buf.to_string_lossy(),
        ])
        .status()
        .unwrap();
    assert!(status.success());
}
```

Figure E.1: build.rs file

```
pub fn set_to_unhashed_number(&mut self) {
    use core::str::FromStr;
    let new_number = U256::from_str(NUMBER).unwrap_or_default();
    self.set_number(new_number);
}
```

Figure E.2: Function to add to the contract added by cargo stylus new counter

```
include!(concat!(env!("OUT_DIR"), "/number.rs"));
```

Figure E.3: Other code to add to the same contract

```
#!/bin/bash

set -x

cast call \
  --private-key 0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c22138887752191c9520659 \
  --rpc-url=http://localhost:8547 "$1" \
  'number()(uint256)'

cast send \
  --private-key 0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c22138887752191c9520659 \
  --rpc-url=http://localhost:8547 "$1" \
  'setToUnhashedNumber()'
```



```
cast call \  
  --private-key 0xb6b15c8cb491557369f3c7d2c287b053eb229daa9c22138887752191c9520659 \  
  --rpc-url=http://localhost:8547 "$1" \  
  'number()(uint256)'
```

Figure E.4: Script that can be used to test the modified contract

F. Proof-of-Concept Code for TOB-STYLUSSDK-16

This appendix contains proof-of-concept code for **TOB-STYLUSSDK-16**. In particular, the example code shows that if not explicitly cleared, the value returned by `shrink` will remain in storage.

The example pushes the value 99 onto a storage vector and then calls `shrink` on that vector. Thereafter, the code calls `grow`, which returns an accessor to the newly created storage element. Printing that element shows that it still contains 99.

```
stylus_sdk::console!("self.vec.len() = {}", self.vec.len()); // prints 0
self.vec.push(U256::from(99));

stylus_sdk::console!("self.vec.len() = {}", self.vec.len()); // prints 1

let x = self.vec.shrink();

stylus_sdk::console!("x = {:?}", x.as_ref().map(|x| x.get())); // prints Some(99)
stylus_sdk::console!("self.vec.len() = {}", self.vec.len()); // prints 0

let y = self.vec.grow();

stylus_sdk::console!("y = {:?}", y.get()); // prints 99
stylus_sdk::console!("self.vec.len() = {}", self.vec.len()); // prints 1
```

Figure F.1: Code to demonstrate that, if not explicitly cleared, the value returned by `shrink` will remain in storage

G. Proof-of-Concept Code for TOB-STYLUSSDK-18

This appendix contains proof-of-concept code for **TOB-STYLUSSDK-18**. Deploying the StylusTest contract and calling its `div0` method results in a division-by-zero panic, causing the transaction to revert.

```
pub trait MyTrait {
    const NAME: &'static str;
}

sol_storage! {
    pub struct MyStruct<T> {
        StorageVec<PhantomData<T>> phantom;
    }
}

impl<T: MyTrait> MyStruct<T> {
    pub fn div0(&mut self) {
        let y = self.phantom.grow();
    }
}

pub struct StylusTest;
impl MyTrait for StylusTest {
    const NAME: &'static str = "StylusTest";
}

sol_storage! {
    #[entrypoint]
    pub struct Test {
        MyStruct<StylusTest> t;
    }
}

#[public]
impl Test {
    pub fn div0(&mut self) {
        self.t.div0();
    }
}
```

Figure G.1: Code to demonstrate a division by zero when using `StorageVec<PhantomData<T>>`

H. Proof-of-Concept Code for TOB-STYLUSSDK-23

This appendix contains proof-of-concept code for **TOB-STYLUSSDK-23**. This smart contract cannot be compiled with cargo build since the AbiType macro will create a selector ABI longer than 1024 characters, leading to a panic.

```
#![cfg_attr(not(any(test, feature = "export-abi")), no_main)]

extern crate alloc;
use alloy_sol_types::sol;
use stylus_sdk::abi::AbiType;
use stylus_sdk::prelude::*;
sol! {
    #[derive(AbiType)]
    struct Person {
        string firstName;
        string middleName;
        string age;
        address homeAddress;
        address lastName;
        address targetAddress;
        address otherAddress;
    }

    #[derive(AbiType)]
    struct Partnership {
        Person primaryPartner;
        Person secondaryPartner;
    }

    #[derive(AbiType)]
    struct Household {
        Partnership residents;
    }

    #[derive(AbiType)]
    struct DepartmentList {
        Partnership[] leadership;
    }

    #[derive(AbiType)]
    struct Division {
        DepartmentList operations;
        DepartmentList administration;
    }

    #[derive(AbiType)]
    struct Corporation {
        Division headquarters;
        Division northRegion;
        Division firstName;
    }
}
```


About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Offchain Labs under the terms of the project statement of work and has been made public at Offchain Labs' request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.